LeetCode Practice

23 Medium: DFS

Question, examples, then the six-phase interview approach with the brute force in phase 2

A split copy of the full All Questions book; every feature and link works the same

23
MEDIUM

Patterns (in order)

1. DFS (23)

How to use

- Tap any title in the Contents to jump to it. A topic name opens that topic's own divider page.
- Every page has < Prev | Contents | Next > at the top.
- Prev and Next move one question at a time; Contents returns here. Each topic starts on its own page.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

23 questions | tap a title to jump to it

Medium questions (23)	Go >
DFS (23)	**Go >**
M 1. Surrounded Regions	LC 130
M 2. Clone Graph	LC 133
M 3. Number of Islands	LC 200
M 4. Course Schedule	LC 207
M 5. Course Schedule II	LC 210
M 6. Graph Valid Tree	LC 261
M 7. Minimum Height Trees	LC 310
M 8. Number of Connected Components in an Undirected Graph	LC 323
M 9. Evaluate Division	LC 399
M 10. Pacific Atlantic Water Flow	LC 417
M 11. The Maze	LC 490
M 12. Number of Provinces	LC 547
M 13. Redundant Connection	LC 684
M 14. Number of Distinct Islands	LC 694
M 15. Max Area of Island	LC 695
M 16. Accounts Merge	LC 721
M 17. Is Graph Bipartite?	LC 785
M 18. Find Eventual Safe States	LC 802
M 19. Most Stones Removed with Same Row or Column	LC 947
M 20. Number of Enclaves	LC 1020
M 21. Web Crawler	LC 1236
M 22. Web Crawler Multithreaded	LC 1242
M 23. Course Schedule IV	LC 1462

Medium questions (23)

MEDIUM | TOPIC 1 OF 1

DFS

23 Medium questions start on the next page

Before this: this is the first topic
After this: this is the last topic

IN THIS TOPIC

1. Surrounded Regions LC 130
2. Clone Graph LC 133
3. Number of Islands LC 200
4. Course Schedule LC 207
5. Course Schedule II LC 210
6. Graph Valid Tree LC 261
7. Minimum Height Trees LC 310
8. Number of Connected Components in an Undirected Graph LC 323
9. Evaluate Division LC 399
10. Pacific Atlantic Water Flow LC 417
11. The Maze LC 490
12. Number of Provinces LC 547
13. Redundant Connection LC 684
14. Number of Distinct Islands LC 694
15. Max Area of Island LC 695
16. Accounts Merge LC 721
17. Is Graph Bipartite? LC 785
18. Find Eventual Safe States LC 802
19. Most Stones Removed with Same Row or Column LC 947
20. Number of Enclaves LC 1020
21. Web Crawler LC 1236
22. Web Crawler Multithreaded LC 1242
23. Course Schedule IV LC 1462

MEDIUM | DFS | LeetCode #130 | 1 of 23 | 1/23

Surrounded Regions

QUESTION

You are given an m x n matrix board containing letters 'X' and 'O', capture regions that are surrounded:

CONSTRAINTS

- Connect: A cell is connected to adjacent cells horizontally or vertically.
- Region: To form a region connect every 'O' cell.
- Surround: A region is surrounded if none of the 'O' cells in that region are on the edge of the board. Such regions are completely enclosed by 'X' cells.

To capture a surrounded region, replace all 'O's with 'X's in-place within the original board. You do not need to return anything.

Example 1

Input	`board = [["X","X","X","X"],["X","O","O","X"],["X","X","O","X"],["X","O","X","X"]]`
Output	`[["X","X","X","X"],["X","X","X","X"],["X","X","X","X"],["X","O","X","X"]]`
Explanation	*In the above diagram, the bottom region is not captured because it is on the edge of the board and cannot be surrounded.*

Example 2

Input	`board = [["X"]]`
Output	`[["X"]]`

- m == board.length
- n == board[i].length
- 1 <= m, n <= 200
- board[i][j] is 'X' or 'O'.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Capture surrounded 'O' regions by replacing them with 'X'.

A region is NOT captured if any 'O' in it touches the board edge. Right?

In-place modification — no return value."

"Border-connected 'O's and their inland 'O' neighbours are safe. Everything else gets flipped."

Is the graph guaranteed to be connected? Can it contain cycles or self-loops?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for cell in cells:
    DFS its region to see if any cell touches the
        border
    if none does:
        flip the region to 'X'
```

TIME	SPACE
O((m * n)^2)	O(m * n)

WHY IT IS SLOW

Re-explores the same region from every cell in it. Start one DFS from each border 'O', mark the safe cells, then flip the rest: O(m * n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE (reverse: mark safe, then flip)

"Border-DFS to mark safe cells, then second pass to flip — already O(mn). Walk-through:"

Step 1 — Mark safe cells:

Scan all 4 borders. For each 'O' on a border: DFS to mark it and all connected 'O's as '#'.

`dfs(r,c): if out of bounds or not 'O': return. grid[r][c]='#'. Recurse in 4 directions.`

WHY '#': temporarily marks 'O's reachable from border as safe (don't capture them).

Step 2 — Flip:

Scan entire grid:

'O' -> 'X' (inner 'O' surrounded -> captured)

'#' -> 'O' (border-connected -> restore to 'O')

'X' -> 'X' (wall, unchanged)

WHY reverse approach: it's much easier to find 'O's connected to the border (safe)

than to find 'O's NOT connected to it (surrounded). Mark the easy case, flip everything else.

Time: O(mn). Space: O(mn) recursion."

Time: O(V+E) — visit each vertex and edge exactly once. Optimal DFS/BFS.

PHASE 4 CLEAN CODE

```
class _130_SurroundedRegions:
    def solve(self, board: List[List[str]]) -> None:
        m, n = len(board), len(board[0])

        def mark_safe(r, c):
            if r < 0 or r >= m or c < 0 or c >= n or board
[r][c] != 'O':
                return
            board[r][c] = '#'
            for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)]:
                mark_safe(r + dr, c + dc)
```

```
        for r in range(m):
            for c in [0, n - 1]:
                mark_safe(r, c)
        for c in range(n):
            for r in [0, m - 1]:
                mark_safe(r, c)

        for r in range(m):
            for c in range(n):
                if board[r][c] == 'O':   board[r][c] = 'X'
                elif board[r][c] == '#': board[r][c] = 'O'
```

PHASE 5 TEST & DEBUG

"Let me trace the two-pass border-marking approach on the classic example."

[["X","X","X","X"],["X","O","O","X"],["X","X","O","X"],["X","O","X","X"]]:

Pass 1 — start DFS only from border cells. The only border 'O' is at (3,1), so mark it '#'.

The inner O's at (1,1),(1,2),(2,2) are NOT connected to a border O, so they stay 'O' for now.

Board after pass 1: inner region still O, border-connected cell is '#'.

Pass 2 — flip all remaining 'O' -> 'X', then restore '#' -> 'O'.

Result: captured center region, bottom border cell stays O ok

[["X","O","X"],["X","O","X"],["X","O","X"]]:

Border DFS from (0,1) marks the whole vertical strip: (0,1)->(1,1)->(2,1) as '#'.

Pass 2 restores all '#' back to 'O', so nothing is captured ok

WHY border DFS is the key insight: only O's disconnected from the border should flip.

Mark safe border-connected O's first, then every leftover O is guaranteed to be captured.

DFS flood-fill component: border-connected O cells = one component (marked '#'). Interior O cells = isolated components. Flip all unmarked O->X. component count=border+interior groups.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(m*n)	**O(m*n)** stack. The reverse-DFS from borders is the key insight —

instead of deciding which 'O's are captured, decide which are SAFE and flip the rest.

Given more time I'd ask: what if the grid wraps (toroidal)? No borders -> all 'O's are safe.

MEDIUM | DFS | LeetCode #133 | 2 of 23 | 2/23

Clone Graph

QUESTION

Given a reference of a node in a connected undirected graph.

Return a deep copy (clone) of the graph.

Each node in the graph contains a value (int) and a list (List[Node]) of its neighbors.

```
class Node {
    public int val;
    public List<Node> neighbors;
}
```

Test case format:

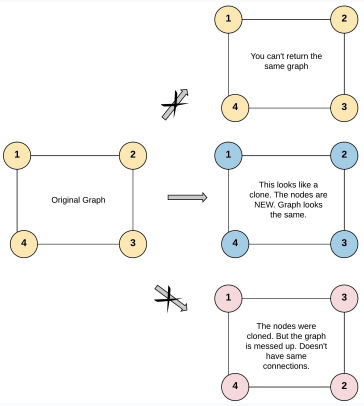
For simplicity, each node's value is the same as the node's index (1-indexed). For example, the first node with val == 1, the second node with val == 2, and so on. The graph is represented in the test case using an adjacency list.

An adjacency list is a collection of unordered lists used to represent a finite graph. Each list describes the set of neighbors of a node in the graph.

The given node will always be the first node with val = 1. You must return the copy of the given node as a reference to the cloned graph.

Example 1

Input	`adjList = [[2,4],[1,3],[2,4],[1,3]]`
Output	`[[2,4],[1,3],[2,4],[1,3]]`
Explanation	*There are 4 nodes in the graph.* *1st node (val = 1)'s neighbors are 2nd node (val = 2) and 4th node (val = 4).* *2nd node (val = 2)'s neighbors are 1st node (val = 1) and 3rd node (val = 3).* *3rd node (val = 3)'s neighbors are 2nd node (val = 2) and 4th node (val = 4).* *4th node (val = 4)'s neighbors are 1st node (val = 1) and 3rd node (val = 3).*



QUESTION

Example 2

Input	`adjList = [[]]`
Output	`[[]]`
Explanation	*Note that the input contains one empty list. The graph consists of only one node with val = 1 and it does not have any neighbors.*



Example 3

Input	`adjList = []`
Output	`[]`
Explanation	*This an empty graph, it does not have any nodes.*

CONSTRAINTS

- The number of nodes in the graph is in the range [0, 100].
- 1 <= Node.val <= 100
- Node.val is unique for each node.
- There are no repeated edges and no self-loops in the graph.
- The Graph is connected and all nodes can be visited starting from the given node.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Deep copy an undirected connected graph. No shared references with original. Right?

Graphs can have cycles — unlike trees, so we need a visited map."

Should I handle disconnected graphs or graphs with no edges?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
nodes = a BFS list of every node
make one new node per original
for neighbour in neighbours:
    find the neighbour's copy by scanning the
        list and connect them
```

TIME	SPACE
O(V * E)	O(V)

PHASE 2 BRUTE FORCE

WHY IT IS SLOW

Finding a copy by scanning costs O(V) per edge. A hash map from original node to its copy, filled while traversing, makes every lookup O(1): O(V + E).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"DFS with a visited map IS already optimal — O(V+E). No further optimization possible."

Walk-through of how the recursion generates the clone (critical for interviews):

clone(node):

1. If node already in cloned -> return cloned[node] (cycle detection — don't recurse again).

2. Create copy = Node(node.val).

3. IMMEDIATELY put cloned[node] = copy BEFORE recursing into neighbors.

WHY: if we recurse first, we'll hit the same node again (undirected graph -> cycles)

and recurse infinitely. The pre-assignment breaks the cycle.

4. copy.neighbors = [clone(nb) for nb in node.neighbors]

This recursively clones every neighbor, which in turn clones THEIR neighbors.

5. Return copy — the caller adds it to their neighbors list.

The result: every node is cloned exactly once, every edge is wired exactly twice (undirected).

PHASE 4 CLEAN CODE

```
class _133_CloneGraph:
    def cloneGraph(self, node: 'Node') -> 'Node':
        if not node:
            return None
        cloned: dict = {}

        def clone(n):
            if n in cloned:
                return cloned[n]
            copy = Node(n.val)
            cloned[n] = copy              # mark before rec
ursing to handle cycles
            copy.neighbors = [clone(nb) for nb in n.neighb
ors]
            return copy

        return clone(node)
```

PHASE 5 TEST & DEBUG

"Let me trace cloneGraph on a 4-node cycle and watch the clone map stop repeated work."

Start at node 1:

clone(1): create clone[1]. To fill its neighbors, recurse into 2 and 4.

clone(2): create clone[2]. Its neighbors are 1 and 3.

PHASE 5 TEST & DEBUG

clone(1): already in cloned map -> return existing clone immediately ok

clone(3): create clone[3]. Its neighbors are 2 and 4.

clone(2): already cloned -> reuse it.

clone(4): create clone[4]. Its neighbors 1 and 3 are already cloned -> reuse both.

Now unwind: clone[4] completes, then clone[3], then clone[2], then clone[1].

Every original node got exactly one clone object, and cycle edges point to cloned neighbors, not originals ok

Edge: node=None -> return None immediately ok

WHY the hashmap matters: it does two jobs at once — prevents infinite recursion on cycles

and guarantees shared neighbors point to the same cloned node object.

DFS clone: visited={1:Node(1),2:Node(2),3:Node(3),4:Node(4)}. dfs(node): if node in visited return clone. Push neighbors onto call stack. visited prevents re-cloning.

Graph clone: component = full connected graph (all nodes reachable). find(node)->clone in visited map. cost=O(V+E). union of adjacency list rebuilt from visited.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(V+E)	**O(V)** for the visited map. Mark node before recursing into neighbours —

without this, cycles cause infinite recursion. Same visited-map pattern handles any graph.

Given more time I'd ask: clone a directed graph? Same approach — just don't add reverse edges.

MEDIUM | DFS | LeetCode #200 | 3 of 23 | 3/23

Number of Islands

QUESTION

Given an m x n 2D binary grid grid which represents a map of '1's (land) and '0's (water), return the number of islands.

An island is surrounded by water and is formed by connecting adjacent lands horizontally or vertically. You may assume all four edges of the grid are all surrounded by water.

Example 1

Input	`grid = [ ["1","1","1","1","0"], ["1","1","0","1","0"], ["1","1","0","0","0"], ["0","0","0","0","0"] ]`
Output	`1`

Example 2

Input	`grid = [ ["1","1","0","0","0"], ["1","1","0","0","0"], ["0","0","1","0","0"], ["0","0","0","1","1"] ]`
Output	`3`

CONSTRAINTS

- m == grid.length
- n == grid[i].length
- 1 <= m, n <= 300
- grid[i][j] is '0' or '1'.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Count connected components of '1's in a binary grid. Diagonal connections don't count. Right?"

Is the graph guaranteed to be connected? Can it contain cycles or self-loops?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
count = 0
for cell in cells:
    if it is unvisited land:
        count = count + 1 and recursively
            mark all land connected to it as
            visited
```

TIME	SPACE
O(m * n)	O(m * n)

PHASE 2 BRUTE FORCE

WHY IT IS SLOW

Time is already optimal. The extra cost is the visited grid and a recursion that can be m * n deep. Sinking visited land in place, with BFS or union-find, avoids the depth and the visited array.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

*"DFS flood-fill IS already the optimal O(m*n) approach."*

Walk-through of how DFS generates the island count:

For each unvisited '1': call dfs(r, c).

`dfs(r, c):`

Base cases: out of bounds OR cell is not '1' -> return immediately.

Mark grid[r][c] = '#' (visited in-place — avoids a separate visited set).

Recurse in all 4 directions: dfs(r-1,c), dfs(r+1,c), dfs(r,c-1), dfs(r,c+1).

Each dfs call floods one connected island component completely before returning.

After dfs returns, we increment count -> one full island discovered.

In-place marking means each cell is visited at most once across ALL dfs calls -> O(m*n) total.

PHASE 4 CLEAN CODE

```
class _200_NumberOfIslands:
    def numIslands(self, grid: List[List[str]]) -> int:
        m, n = len(grid), len(grid[0])

        def dfs(r, c):
            if r < 0 or r >= m or c < 0 or c >= n or grid[
r][c] != '1':
                return
            grid[r][c] = '#'    # mark visited by flooding
            for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)]:
                dfs(r + dr, c + dc)

        count = 0
        for r in range(m):
            for c in range(n):
                if grid[r][c] == '1':
                    dfs(r, c)
                    count += 1
        return count
```

PHASE 5 TEST & DEBUG

"Let me trace numIslands on [['1','1','0'],['0','1','0'],['0','0','1']]."

DFS from each unvisited '1'. Mark visited by setting to '0'.

(0,0)='1' -> dfs(0,0): mark '0'. Visit (0,1)='1'->dfs(0,1): mark '0'. (1,1)='1'->dfs(1,1): mark '0'. All neighbors of (1,1) are '0' or out of bounds -> return. Back to (0,1): no more live neighbors. count=1.

(0,2)='0' skip. (1,0)='0' skip. (1,1)='0' skip. (1,2)='0' skip.

(2,0)='0' skip. (2,1)='0' skip. (2,2)='1' -> dfs(2,2): mark '0'. count=2.

Return 2 ok

Edge: all water -> return 0 ok. All land -> DFS spreads to entire grid -> 1 island ok

PHASE 5 TEST & DEBUG

WHY mark '0' during DFS: prevents revisiting. No extra visited array needed (in-place O(1) space).

DFS: visited (mark cell '0' to sink it). island_count=0. For each unvisited '1': dfs->sink all connected land. visited via mutation: '1'->'0'. component per call=1 island.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(m*n)	**O(m*n)** stack. Flooding in-place avoids a separate visited set

BFS alternative: use a queue instead of recursion — better for very large grids (no stack overflow).

Union-Find alternative: O(m*n * alpha(m*n)) — useful when islands merge dynamically.

Given more time I'd ask: restore the grid after counting?

Keep a separate visited set or restore '#' -> '1' after each DFS.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | DFS | LeetCode #207 | 4 of 23 | 4/23

Course Schedule

QUESTION

There are a total of numCourses courses you have to take, labeled from 0 to numCourses - 1. You are given an array prerequisites where prerequisites[i] = [a_i, b_i] indicates that you must take course b_i first if you want to take course a_i.

CONSTRAINTS

- For example, the pair [0, 1], indicates that to take course 0 you have to first take course 1.

Return true if you can finish all courses. Otherwise, return false.

Example 1

Input	`numCourses = 2, prerequisites = [[1,0]]`
Output	`true`
Explanation	*There are a total of 2 courses to take. To take course 1 you should have finished course 0. So it is possible.*

Example 2

Input	`numCourses = 2, prerequisites = [[1,0],[0,1]]`
Output	`false`
Explanation	*There are a total of 2 courses to take. To take course 1 you should have finished course 0, and to take course 0 you should also have finished course 1. So it is impossible.*

- 1 <= numCourses <= 2000
- 0 <= prerequisites.length <= 5000
- prerequisites[i].length == 2
- 0 <= a_i, b_i < numCourses
- All the pairs prerequisites[i] are unique.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Can we finish all n courses given prerequisite pairs [a,b] meaning b must come before a?

Equivalent to: does the directed graph have no cycle? Right?"

"A few quick questions:

Return bool. If a cycle exists, some course can never be taken -> return false."

"Let me walk: n=2, prerequisites=[[1,0]] -> 0 before 1 -> no cycle -> true ok. [[1,0],[0,1]] -> cycle -> false ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for course in courses:
    search along prerequisite edges from it
    if the search comes back to that course there
        is a cycle:
        return False
return True
```

TIME	SPACE
O(V * (V + E))	O(V)

WHY IT IS SLOW

Starts a fresh search from every course and repeats work. Kahn's algorithm (repeatedly remove courses with no remaining prerequisites) or one DFS with three colours is O(V + E).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Kahn's BFS topological sort — already O(V+E). Walk-through of how cycle detection works:"

Step 1: Build adjacency list and compute in-degree (number of incoming edges) for each node.

Step 2: Enqueue all nodes with in-degree == 0 (no prerequisites).

Step 3: BFS loop:

Pop node u. completed += 1.

For each neighbor v of u: indegree[v] -= 1.

If indegree[v] == 0 -> enqueue v (all its prerequisites are now done).

Step 4: If completed == numCourses -> no cycle, all courses can be taken. Else -> cycle detected.

WHY cycle detection works: nodes in a cycle NEVER reach indegree 0 (they depend on each other),

so they're never enqueued -> never counted -> completed < numCourses.

Time: O(V+E). Space: O(V+E).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
from collections import defaultdict, deque
class _207_CourseSchedule:
    def canFinish(self, numCourses, prerequisites):
        graph = defaultdict(list)
        indegree = [0] * numCourses
        for a, b in prerequisites:
            graph[b].append(a)
            indegree[a] += 1
        queue = deque(i for i in range(numCourses) if inde
gree[i] == 0)
        processed = 0
        while queue:
```

```
            node = queue.popleft()
            processed += 1
            for neighbor in graph[node]:
                indegree[neighbor] -= 1
                if indegree[neighbor] == 0:
                    queue.append(neighbor)
        return processed == numCourses
```

PHASE 5 TEST & DEBUG

"Let me trace canFinish on n=4, prerequisites=[[1,0],[2,0],[3,1],[3,2]]."

Build graph: 0->[1,2], 1->[3], 2->[3].

Kahn's BFS: in-degree=[0,1,1,2]. queue=[0]. processed=0.

Pop 0: neighbors 1,2. in_degree[1]--=0->enqueue 1. in_degree[2]--=0->enqueue 2. processed=1.

Pop 1: neighbor 3. in_degree[3]--=1. processed=2.

Pop 2: neighbor 3. in_degree[3]--=0->enqueue 3. processed=3.

Pop 3: no neighbors. processed=4. 4==n -> True ok

Cycle case [[1,0],[0,1]]: in_degree=[1,1]. queue=[]. processed=0!=2 -> False ok

WHY processed==n: every course was reachable — no cycle trapped any node.

DFS cycle detect: visited[node]=1 (in current path), 2 (done). call stack: dfs(0)->dfs(1)->dfs(0): finds visited[0]=1 -> CYCLE -> False. pop stack, mark done.

DAG (Directed Acyclic Graph) check: component with cycle->course impossible. dist[node]=topo order. If back-edge found during DFS -> cycle -> return False. Reverse topo = valid course order.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(V+E)	O(V+E)

DFS approach also O(V+E) — Kahn's is iterative (no stack overflow risk for large graphs).

Follow-up: Course Schedule II (#210) — same Kahn's BFS, collect the order.

Follow-up: Minimum Semesters (#1136) — count BFS levels to get minimum time.

MEDIUM | DFS | LeetCode #210 | 5 of 23 | 5/23

Course Schedule II

QUESTION

There are a total of numCourses courses you have to take, labeled from 0 to numCourses - 1. You are given an array prerequisites where prerequisites[i] = [a_i, b_i] indicates that you must take course b_i first if you want to take course a_i.

CONSTRAINTS

- For example, the pair [0, 1], indicates that to take course 0 you have to first take course 1.

Return the ordering of courses you should take to finish all courses. If there are many valid answers, return any of them. If it is impossible to finish all courses, return an empty array.

Example 1

Input	`numCourses = 2, prerequisites = [[1,0]]`
Output	`[0,1]`
Explanation	*There are a total of 2 courses to take. To take course 1 you should have finished course 0. So the correct course order is [0,1].*

Example 2

Input	`numCourses = 4, prerequisites = [[1,0],[2,0],[3,1],[3,2]]`
Output	`[0,2,1,3]`
Explanation	*There are a total of 4 courses to take. To take course 3 you should have finished both courses 1 and 2. Both courses 1 and 2 should be taken after you finished course 0. So one correct course order is [0,1,2,3]. Another correct ordering is [0,2,1,3].*

Example 3

Input	`numCourses = 1, prerequisites = []`
Output	`[0]`

- 1 <= numCourses <= 2000
- 0 <= prerequisites.length <= numCourses * (numCourses - 1)
- prerequisites[i].length == 2
- 0 <= a_i, b_i < numCourses
- a_i != b_i
- All the pairs [a_i, b_i] are distinct.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Return a valid course ordering (topological sort). Return [] if no valid order (cycle exists). Right?"

"A few quick questions:

Any valid topological order is acceptable? Yes."

"Let me walk: n=4, prerequisites=[[1,0],[2,0],[3,1],[3,2]] -> [0,1,2,3] or [0,2,1,3] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for ordering in orderings(courses):
    if every prerequisite comes before its course:
        return it
```

TIME	SPACE
O(n! * p)	O(n)

WHY IT IS SLOW

n! orderings to test. A topological sort (Kahn's queue of zero-in-degree courses) or DFS post-order is O(V + E).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Kahn's BFS — same as #207 but collect order instead of just counting. Already O(V+E)."

Walk-through:

Same Kahn's BFS as #207. Additionally, append each dequeued node to 'order'.

When BFS completes: if len(order) == numCourses -> return order (valid topological order).

Else -> cycle exists -> return [].

WHY the dequeue ORDER gives a valid schedule: we only dequeue a node after ALL its

prerequisites have been dequeued — so prerequisites always appear BEFORE the course.

Edge case: multiple valid orderings exist (depends on which 0-indegree node we pick first);

any valid one is acceptable.

Time: O(V+E). Space: O(V+E)."

Time: O(log n) — binary search on answer space.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
from collections import defaultdict, deque
class _210_CourseScheduleII:
    def findOrder(self, numCourses, prerequisites):
        graph = defaultdict(list)
        indegree = [0] * numCourses
        for course, prereq in prerequisites:
            graph[prereq].append(course)
            indegree[course] += 1
        queue = deque(i for i in range(numCourses) if inde
gree[i] == 0)
        order = []
        while queue:
            node = queue.popleft()
            order.append(node)
            for neighbor in graph[node]:
                indegree[neighbor] -= 1
                if indegree[neighbor] == 0:
                    queue.append(neighbor)
        return order if len(order) == numCourses else []
```

PHASE 5 TEST & DEBUG

"Let me trace findOrder on n=4, prerequisites=[[1,0],[2,0],[3,1],[3,2]]."

Same as Q207 but collect the order.

in_degree=[0,1,1,2]. queue=[0]. order=[].

Pop 0->order=[0]. Decrement neighbors 1,2. in_degree=[0,0,0,2].

Enqueue 1,2. Pop 1->order=[0,1]. Decrement 3->in_degree[3]=1.

Pop 2->order=[0,1,2]. Decrement 3->in_degree[3]=0. Enqueue 3.

Pop 3->order=[0,1,2,3]. Return [0,1,2,3] ok

Cycle case: order shorter than n -> return [] ok

WHY BFS gives topological order: each popped node has all prerequisites already done (in_degree=0).

DFS topo sort: visited marks nodes 0=unvisited,1=in-stack,2=done. call stack: dfs(0)->dfs(1)->dfs(2)->push 2 done. Result=reverse order of finish. visited[2]=2 ok

DAG (Directed Acyclic Graph): component=connected course group. dist[node]=finish-time in DFS. find(node)->topo position. Stack order after DFS = valid course schedule.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(V+E)	O(V+E)

Identical to Course Schedule I (#207) — just collect nodes instead of counting.

Follow-up: Alien Dictionary (#269) — same topological sort on character ordering.

Follow-up: if multiple valid orders exist, return lex-smallest — use a min-heap.

MEDIUM | DFS | LeetCode #261 | 6 of 23 | 6/23

Graph Valid Tree

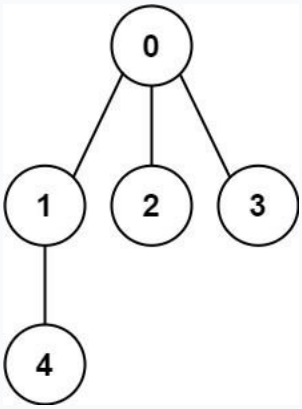
QUESTION

You have a graph of n nodes labeled from 0 to n - 1. You are given an integer n and a list of edges where edges[i] = [a_i, b_i] indicates that there is an undirected edge between nodes a_i and b_i in the graph.

Return true if the edges of the given graph make up a valid tree, and false otherwise.

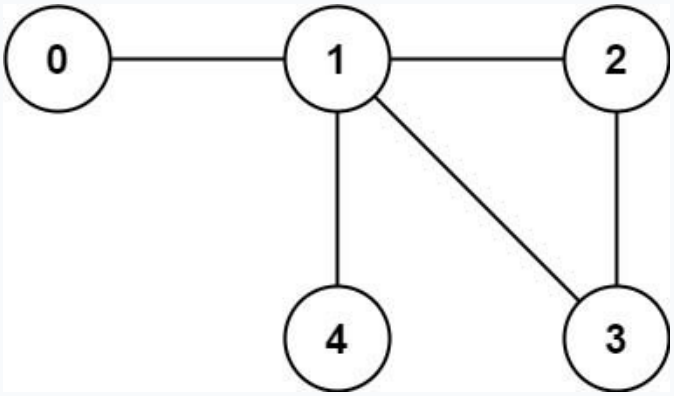
Example 1

Input	`n = 5, edges = [[0,1],[0,2],[0,3],[1,4]]`
Output	`true`



Example 2

Input	`n = 5, edges = [[0,1],[1,2],[2,3],[1,3],[1,4]]`
Output	`false`



CONSTRAINTS

- 1 <= n <= 2000
- 0 <= edges.length <= 5000
- edges[i].length == 2
- 0 <= a_i, b_i < n
- a_i != b_i
- There are no self-loops or repeated edges.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Valid tree: connected, acyclic, exactly n-1 edges. Right?

Any two of these imply the third for n nodes."

Can the tree be empty (root=null)? Are all node values unique? Right?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for v) in v)s:
    check with a fresh DFS whether u and v are
        already connected (then there is a cycle)
    else:
        add the edge
    at the end check the graph is one component
```

TIME	SPACE
O(V * E)	O(V)

WHY IT IS SLOW

A new graph search per edge. Union-find (a cycle exists when both endpoints already share a root) with the n - 1 edge check is nearly O(E).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"DFS with parent tracking IS already O(n) — optimal."

How the algorithm proves tree validity:

Condition 1 (fast filter): len(edges) != n-1 -> immediately False.

A tree of n nodes has exactly n-1 edges. Too few -> disconnected. Too many -> cycle.

Condition 2 (DFS connectivity check):

`dfs(node, parent): add to visited, recurse on all neighbors EXCEPT parent.`

Pass parent to avoid treating the edge we came from as a back-edge.

If we reach a VISITED neighbor (that isn't parent) -> cycle detected -> return False.

After DFS: if len(visited) == n -> all nodes reached -> connected.

Both conditions together valid tree.

PHASE 4 CLEAN CODE

```
class _261_GraphValidTree:
    def validTree(self, n: int, edges: List[List[int]]) ->
 bool:
        if len(edges) != n - 1:
            return False
        graph   = defaultdict(list)
        for a, b in edges:
            graph[a].append(b)
            graph[b].append(a)
        visited = set()
```

```
        def dfs(node, parent):
            visited.add(node)
            for nb in graph[node]:
                if nb == parent: continue
                if nb in visited: return False
                if not dfs(nb, node): return False
            return True

        return dfs(0, -1) and len(visited) == n
```

PHASE 5 TEST & DEBUG

"Let me trace the Union-Find version on one valid tree and one cycle case."

n=5, edges=[[0,1],[0,2],[0,3],[1,4]]:

First check: edges=4 and n-1=4, so it passes the necessary edge-count test ok

union(0,1): merge two components.

union(0,2): node 2 joins root 0's component.

union(0,3): node 3 joins the same component.

union(1,4): find(1) compresses to root 0, so node 4 also joins root 0's component.

End state: all nodes have the same root, so the graph is connected -> True ok

Cycle case [[0,1],[1,2],[2,3],[1,3],[1,4]]:

By the time we process (1,3), find(1) == find(3), meaning they are already connected.

Adding this edge would close a cycle, so return False immediately

WHY Union-Find is great here: it lets us detect "already connected" edges in near O(1),

which is exactly the signal that a supposed tree has formed a cycle.

Union-Find: union(0,1)->root=0. union(1,2)->root=0. find(0)==find(2): already same component. Adding edge 0-2 with 2 existing edges and 3 nodes -> cycle -> not a valid tree.

DFS alternative: visited=set(). DFS from node 0: push unvisited neighbors. visited={0,1,2,3}. Valid tree iff all n nodes reachable AND exactly n-1 edges (no cycle via find(u)==find(v)).

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(V+E) =O(n)	**O(n)** Early return if edges!=n-1 is a fast filter

Union-Find alternative: O(n * alpha(n)) — merge edges, detect cycle if already connected.

Given more time I'd ask: disconnected forest (multiple trees)?

Return number of components instead — skip the len(edges)==n-1 check.

MEDIUM | DFS | LeetCode #310 | 7 of 23 | 7/23

Minimum Height Trees

QUESTION

A tree is an undirected graph in which any two vertices are connected by exactly one path. In other words, any connected graph without simple cycles is a tree.

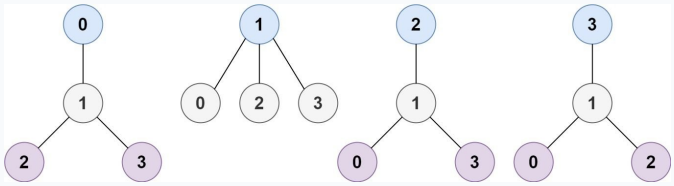
Given a tree of n nodes labelled from 0 to n - 1, and an array of n - 1 edges where edges[i] = [a_i, b_i] indicates that there is an undirected edge between the two nodes a_i and b_i in the tree, you can choose any node of the tree as the root. When you select a node x as the root, the result tree has height h. Among all possible rooted trees, those with minimum height (i.e. min(h)) are called minimum height trees (MHTs).

Return a list of all MHTs' root labels. You can return the answer in any order.

The height of a rooted tree is the number of edges on the longest downward path between the root and a leaf.

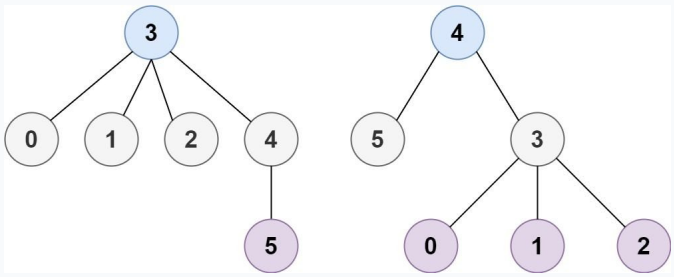
Example 1

Input	`n = 4, edges = [[1,0],[1,2],[1,3]]`
Output	`[1]`
Explanation	*As shown, the height of the tree is 1 when the root is the node with label 1 which is the only MHT.*



Example 2

Input	`n = 6, edges = [[3,0],[3,1],[3,2],[3,4],[5,4]]`
Output	`[3,4]`



CONSTRAINTS

- $1 <= n <= 2 * 10^4$
- edges.length == n - 1
- $0 <= a_i, b_i < n$
- a_i != b_i
- All the pairs (a_i, b_i) are distinct.

QUESTION

- The given input is guaranteed to be a tree and there will be no repeated edges.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Find which nodes make MHT (Minimum Height Tree) roots. The answer is at most 2 nodes — the tree's center(s). Right?

Repeatedly prune leaf nodes (degree 1) until 1 or 2 nodes remain."

"Like peeling an onion from outside in — the core is the answer."

Can the tree be empty (root=null)? Are all node values unique? Right?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for root in roots:
    compute the tree height (BFS or DFS)
return the nodes with the smallest height
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

A full traversal per root. Repeatedly peel off all leaves; the last one or two nodes left are the centres, O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Leaf-pruning BFS IS already O(n) — optimal."

Walk-through of how the centroid emerges:

Think of the tree as an onion. Leaves are the outermost layer.

Round 1: identify all degree-1 nodes (leaves). Remove them.

For each removed leaf, reduce its neighbor's degree. If neighbor becomes degree-1, it's a new leaf.

Repeat: peel the next outer layer of leaves.

Stop when 1 or 2 nodes remain — these are the centroids (can't peel further without cutting the tree).

WHY at most 2? A tree's diameter has 1 midpoint (odd length) or 2 midpoints (even length).

Peeling converges to exactly these midpoints.

PHASE 4 CLEAN CODE

```
class _310_MinHeightTrees:
    def findMinHeightTrees(self, n: int, edges: List[List[
int]]) -> List[int]:
        if n == 1:
            return [0]
        graph  = defaultdict(set)
        for a, b in edges:
            graph[a].add(b)
            graph[b].add(a)
        leaves = [node for node in range(n) if len(graph[n
ode]) == 1]
        remaining = n
        while remaining > 2:
            remaining -= len(leaves)
            new_leaves = []
            for leaf in leaves:
                nb = next(iter(graph[leaf]))
                graph[nb].remove(leaf)
                if len(graph[nb]) == 1:
                    new_leaves.append(nb)
            leaves = new_leaves
        return leaves
```

PHASE 5 TEST & DEBUG

"Let me trace findMinHeightTrees on n=4, edges=[[1,0],[1,2],[1,3]]."

Topological sort / leaf trimming: star graph centered at 1

`leaves = [0,2,3] (degree 1)`

Trim leaves: remaining=[1,0,2,3] -> remove 0,2,3 -> remaining=[1]

n=1 -> only node 1 left -> return [1] ok (root=1 gives height 1)

Why trim: like peeling an onion from outside in; center nodes are always MHT roots.

Trace n=6, edges linear 0-1-2-3-4-5: leaves=[0,5], trim->[1,4], trim->[2,3] -> return [2,3] ok

WHY <=2 centers: any tree has 1 or 2 centroid nodes.

Topo-sort (leaf pruning): queue=[all leaves]. Remove outer ring repeatedly. visited via degree-tracking. Pop leaf, remove from neighbors. component shrinks to 1-2 central nodes ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(n)** This is topological sort on undirected tree — same BFS idea

At most 2 centers exist — if more, tree would have multiple independent long paths.

Given more time I'd ask: all trees of minimum height (not just roots)?

BFS from all roots found, check height — same O(n).

IN YOUR APP | Expose | Editorial | Community

MEDIUM | DFS | LeetCode #323 | 8 of 23 | 8/23

Number of Connected Components in an Undirected Graph

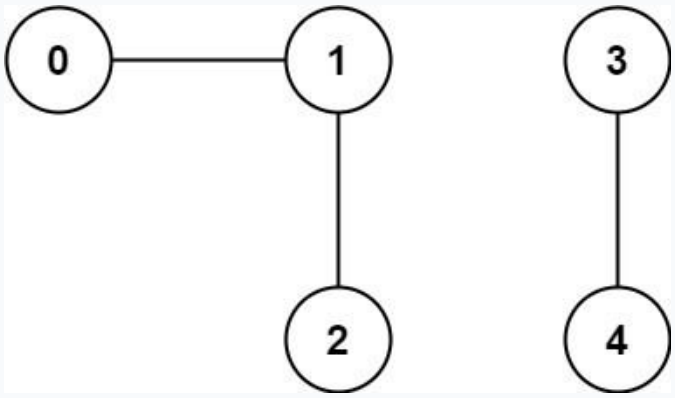
QUESTION

You have a graph of n nodes. You are given an integer n and an array edges where edges[i] = [a_i, b_i] indicates that there is an edge between a_i and b_i in the graph.

Return the number of connected components in the graph.

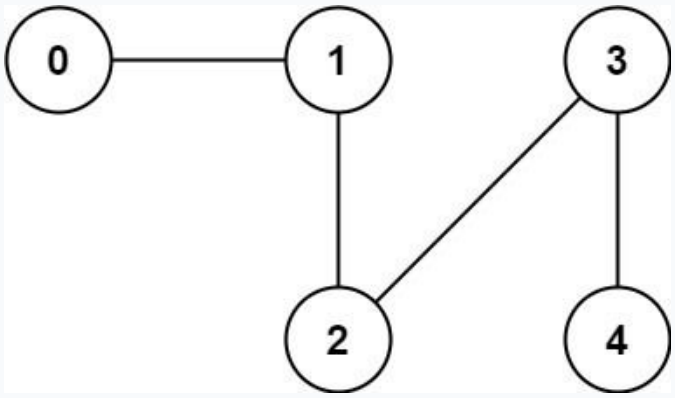
Example 1

Input	`n = 5, edges = [[0,1],[1,2],[3,4]]`
Output	`2`



Example 2

Input	`n = 5, edges = [[0,1],[1,2],[2,3],[3,4]]`
Output	`1`



CONSTRAINTS

- 1 <= n <= 2000
- 1 <= edges.length <= 5000
- edges[i].length == 2
- 0 <= a_i <= b_i < n
- a_i != b_i
- There are no repeated edges.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Count connected components in an undirected graph. Right?"

Is the graph guaranteed to be connected? Can it contain cycles or self-loops?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for (a, b) in pairs(nodes):
    run a fresh BFS to see whether they are connected
    count the groups
```

TIME	SPACE
O(V^2 * (V + E))	O(V)

WHY IT IS SLOW

A search per pair. One DFS per unvisited node (or union-find) labels every component in O(V + E).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"DFS count IS already O(V+E) — optimal for this problem."

How DFS generates the component count:

For each node 0..n-1: if not visited -> we found a new component -> count++, then DFS floods it.

`dfs(node): mark visited, recurse on all unvisited neighbors.`

After dfs returns, the entire component containing 'node' is marked visited.

Next unvisited node MUST be in a different component -> count another.

Union-Find alternative: O(n·alpha(n)) per query — better for dynamic edge additions,

but DFS is simpler to code in an interview for the static case.

PHASE 4 CLEAN CODE

```python
class _323_NumberConnectedComponents:
    def countComponents(self, n: int, edges: List[List[int
]]) -> int:
        graph   = defaultdict(list)
        for a, b in edges:
            graph[a].append(b)
            graph[b].append(a)
        visited = set()

        def dfs(node):
            visited.add(node)
            for nb in graph[node]:
                if nb not in visited:
                    dfs(nb)

        count = 0
        for node in range(n):
            if node not in visited:
                dfs(node)
```

```
            count += 1
        return count
```

PHASE 5 TEST & DEBUG

"Let me trace countComponents on n=5, edges=[[0,1],[1,2],[3,4]]."

Union-Find: initially 5 components.

union(0,1): merge -> 4 components. union(1,2): root(1)=root(0)!=root(2)->merge -> 3 components.

union(3,4): merge -> 2 components.

Return 2 ok (components: {0,1,2} and {3,4}).

DFS alternative: visited=[F]*5. count=0.

i=0: not visited -> dfs(0): visit 0,1,2. count++. i=3: not visited -> dfs(3): visit 3,4. count++.

Return 2 ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(V+E)	**O(V+E)** Standard DFS counting components

Union-Find alternative: merge edges, count unique roots -> O(n * alpha(n)).

Given more time I'd ask: what if edges are added dynamically?

Union-Find with path compression supports O(alpha(n)) per merge query.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | DFS | LeetCode #399 | 9 of 23 | 9/23

Evaluate Division

QUESTION

You are given an array of variable pairs equations and an array of real numbers values, where equations[i] = [A_i, B_i] and values[i] represent the equation A_i / B_i = values[i]. Each A_i or B_i is a string that represents a single variable.

You are also given some queries, where queries[j] = [C_j, D_j] represents the jth query where you must find the answer for C_j / D_j = ?.

Return the answers to all queries. If a single answer cannot be determined, return -1.0.

Note: The input is always valid. You may assume that evaluating the queries will not result in division by zero and that there is no contradiction.

Note: The variables that do not occur in the list of equations are undefined, so the answer cannot be determined for them.

Example 1

Input	`equations = [["a","b"],["b","c"]], values = [2.0,3.0], queries = [["a","c"],["b","a"],["a","e"],["a","a"],["x","x"]]`
Output	`[6.00000,0.50000,-1.00000,1.00000,-1.00000]`
Explanation	*Given:* *a / b = 2.0* *,* *b / c = 3.0* *queries are:* *a / c = ?* *,* *b / a = ?* *,* *a / e = ?* *,* *a / a = ?* *,* *x / x = ?* *return: [6.0, 0.5, -1.0, 1.0, -1.0]* *note: x is undefined => -1.0*

Example 2

Input	`equations = [["a","b"],["b","c"],["bc","cd"]], values = [1.5,2.5,5.0], queries = [["a","c"],["c","b"],["bc","cd"],["cd","bc"]]`
Output	`[3.75000,0.40000,5.00000,0.20000]`

QUESTION

Example 3

Input	`equations = [["a","b"]], values = [0.5], queries = [["a","b"],["b","a"],["a","c"],["x","y"]]`
Output	`[0.50000,2.00000,-1.00000,-1.00000]`

CONSTRAINTS

- 1 <= equations.length <= 20
- equations[i].length == 2
- 1 <= A_i.length, B_i.length <= 5
- values.length == equations.length
- 0.0 < values[i] <= 20.0
- 1 <= queries.length <= 20
- queries[i].length == 2
- 1 <= C_j.length, D_j.length <= 5
- A_i, B_i, C_j, D_j consist of lower case English letters and digits.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Given division equations with results, answer queries. Return -1.0 if a query can't be answered. Right?"

"A few quick questions:

Build a graph where A/B=k means edge A->B with weight k and B->A with weight 1/k.

Query A/B = product of weights along path from A to B? Yes."

"Let me walk: equations=[['a','b'],['b','c']], values=[2.0,3.0], queries=[['a','c']] -> 6.0 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for query in queries:
    DFS every simple path from the numerator to
        the denominator multiplying edge ratios
        until one reaches it
```

TIME	SPACE
O(q * V!)	O(V)

WHY IT IS SLOW

Explores path after path without remembering visited variables globally. A BFS or DFS with a visited set is O(V + E) per query; union-find with weights, or Floyd-Warshall once, answers queries almost instantly.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

*"Union-Find with weighted ratios — O((n+q) * alpha(n)). Walk-through:"*

For each equation a/b=v: union(a, b) with weight v.

Union-Find with weights: parent[node] = (root, weight_to_root).

find(x): path-compress with cumulative weight multiplication:

If parent[x]==x: return (x, 1.0).

root, w = find(parent[x]). parent[x]=(root, w*weight[x]). return (root, w*weight[x]).

union(a, b, val): ra,wa=find(a). rb,wb=find(b). Set parent[ra]=(rb, val*wb/wa).

query(a, b): if unknown var: return -1. ra,wa=find(a). rb,wb=find(b).

If ra!=rb: -1. Else: wa/wb.

WHY weighted union-find: tracks ratio chains (a/c = a/b * b/c) using edge weights.

Time: O((n+q)*alpha(n)). Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
from collections import defaultdict, deque
class _399_EvaluateDivision:
    def calcEquation(self, equations, values, queries):
        graph = defaultdict(dict)
        for (a, b), v in zip(equations, values):
            graph[a][b] = v
            graph[b][a] = 1.0 / v
        def bfs(src, dst):
            if src not in graph or dst not in graph:
                return -1.0
            if src == dst:
                return 1.0
            visited = {src}
            queue = deque([(src, 1.0)])
            while queue:
                node, product = queue.popleft()
                for neighbor, weight in graph[node].items(
):
                    new_product = product * weight
                    if neighbor == dst:
                        return new_product
                    if neighbor not in visited:
                        visited.add(neighbor)
                        queue.append((neighbor, new_produc
t))
            return -1.0
        return [bfs(a, b) for a, b in queries]
```

PHASE 5 TEST & DEBUG

"Let me trace calcEquation on [['a','b'],['b','c']], values=[2,3], queries=[['a','c']]."

Build graph: a->b (weight 2), b->a (1/2), b->c (3), c->b (1/3).

Query a/c: DFS from a to c.

dfs(a, c, visited): try neighbor b (weight=2).

dfs(b, c, visited): try neighbor a (visited, skip). try neighbor c: c==target -> return 3.

return 2 * 3 = 6.

a/c = 6 ok (a/b=2, b/c=3 -> a/c=6).

Query a/a: source==dest -> return 1.0 ok

Query a/e (unknown): dfs reaches dead end -> return -1 ok

WHY DFS with path product: a/c = a/b * b/c = product along path from a to c.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(Q*(V+E))	**O(V+E)**
BFS per query	graph

Precompute all-pairs: O(V³) Floyd-Warshall — better when Q >> V.

Follow-up: update equations dynamically — Union-Find with weighted edges.

Follow-up: if values can be 0, handle division by zero separately.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | DFS | LeetCode #417 | 10 of 23 | 10/23

Pacific Atlantic Water Flow

QUESTION

There is an m x n rectangular island that borders both the Pacific Ocean and Atlantic Ocean. The Pacific Ocean touches the island's left and top edges, and the Atlantic Ocean touches the island's right and bottom edges.

The island is partitioned into a grid of square cells. You are given an m x n integer matrix heights where heights[r][c] represents the height above sea level of the cell at coordinate (r, c).

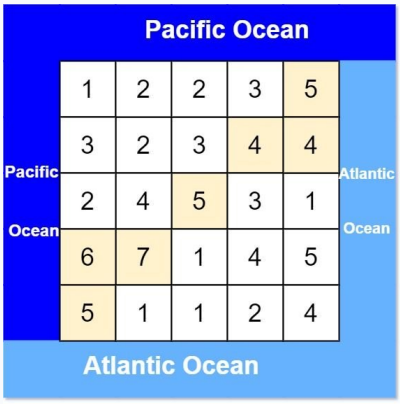
The island receives a lot of rain, and the rain water can flow to neighboring cells directly north, south, east, and west if the neighboring cell's height is less than or equal to the current cell's height. Water can flow from any cell adjacent to an ocean into the ocean.

Return a 2D list of grid coordinates result where result[i] = $[r_i, c_i]$ denotes that rain water can flow from cell (r_i, c_i) to both the Pacific and Atlantic oceans.

Example 1

Input	`heights = [[1,2,2,3,5],[3,2,3,4,4],[2,4,5,3,1],[6,7,1,4,5],[5,1,1,2,4]]`
Output	`[[0,4],[1,3],[1,4],[2,2],[3,0],[3,1],[4,0]]`
Explanation	*The following cells can flow to the Pacific and Atlantic oceans, as shown below:* *[0,4]: [0,4] -> Pacific Ocean* *[0,4] -> Atlantic Ocean* *[1,3]: [1,3] -> [0,3] -> Pacific Ocean* *[1,3] -> [1,4] -> Atlantic Ocean* *[1,4]: [1,4] -> [1,3] -> [0,3] -> Pacific Ocean* *[1,4] -> Atlantic Ocean* *[2,2]: [2,2] -> [1,2] -> [0,2] -> Pacific Ocean* *[2,2] -> [2,3] -> [2,4] -> Atlantic Ocean* *[3,0]: [3,0] -> Pacific Ocean* *[3,0] -> [4,0] -> Atlantic Ocean* *[3,1]: [3,1] -> [3,0] -> Pacific Ocean* *[3,1] -> [4,1] -> Atlantic Ocean* *[4,0]: [4,0] -> Pacific Ocean* *[4,0] -> Atlantic Ocean* *Note that there are other possible paths for these cells to flow to the Pacific and Atlantic oceans.*

QUESTION



Example 2

Input	`heights = [[1]]`
Output	`[[0,0]]`
Explanation	*The water can flow from the only cell to the Pacific and Atlantic oceans.*

CONSTRAINTS

- m == heights.length
- n == heights[r].length
- 1 <= m, n <= 200
- 0 <= heights[r][c] <= 10^5

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Water flows to an adjacent cell if that cell has height <= current cell.

Find cells that can flow to BOTH the Pacific (top/left border) AND Atlantic (bottom/right border). Right?"

"A few quick questions:

Return all such cell coordinates? Yes. Return in any order? Yes."

"Let me walk:
heights=[[1,2,2,3,5],[3,2,3,4,4],[2,4,5,3,1],[6,7,1,4,5],[5,1,1,2,4]] -> [[0,4],[1,3],[1,4],[2,2],[3,0],[3,1],[4,0]] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for cell in cells:
    run a DFS to see whether water can reach the
        Pacific and the Atlantic
    collect the cells that reach both
```

TIME	SPACE
O((m * n)^2)	O(m * n)

WHY IT IS SLOW

PHASE 2 BRUTE FORCE

A fresh search from every cell. Run one BFS or DFS inward from each ocean's border (moving to equal or higher cells) and intersect the two reachable sets: O(m * n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is per-cell BFS.

Reverse BFS from both ocean borders:

From Pacific borders, find all cells that can reach Pacific (flow UPHILL in reverse).

From Atlantic borders, similarly. Intersection = answer.

Time: O(mn). Space: O(mn)."

Time: O(log n) — binary search on answer space.

Key insight: reversing the flow direction — BFS uphill from ocean borders — converts "can this cell reach the ocean?" into "what cells are reachable from the ocean?", reducing two separate BFS sweeps to O(mn).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
from collections import deque
class _417_PacificAtlanticWaterFlow:
    def pacificAtlantic(self, heights):
        m, n = len(heights), len(heights[0])
        def bfs(sources):
            reachable = set(sources)
            queue = deque(sources)
            while queue:
                r, c = queue.popleft()
                for dr, dc in [(0,1),(0,-1),(1,0),(-1,0)]:
                    nr, nc = r + dr, c + dc
                    if (0 <= nr < m and 0 <= nc < n and
                            (nr, nc) not in reachable and
                            heights[nr][nc] >= heights[r][
c]):
                        reachable.add((nr, nc))
                        queue.append((nr, nc))
            return reachable
        pacific = bfs([(0, c) for c in range(n)] + [(r, 0)
 for r in range(m)])
        atlantic = bfs([(m-1, c) for c in range(n)] + [(r,
 n-1) for r in range(m)])
        return [list(cell) for cell in pacific & atlantic]
```

PHASE 5 TEST & DEBUG

"Let me trace pacificAtlantic on [[1,2,2,3,5],[3,2,3,4,4],[2,4,5,3,1],[6,7,1,4,5],[5,1,1,2,4]]."

Pacific (top-left) reachable: BFS/DFS from top row + left col.

Atlantic (bottom-right): BFS from bottom row + right col.

Pacific BFS expands: from all top/left borders, mark cells where water CAN flow to Pacific.

Atlantic BFS: from all bottom/right borders, mark cells where water CAN flow to Atlantic.

Intersection: cells reachable by both.

PHASE 5 TEST & DEBUG

Key: water flows from high to low, but we reverse — from ocean, we climb (low to high or equal).

Result: [[0,4],[1,3],[1,4],[2,2],[3,0],[3,1],[4,0]] ok

WHY reverse BFS: instead of tracking all paths downhill (expensive), mark reachable from ocean uphill.

DFS from pacific: visited_p={(0,0),(0,1),(0,2),(0,3),(0,4)}. DFS from atlantic: visited_a={(4,4),(4,3),...}. Intersection=cells in both visited sets -> result. Each cell visited at most twice.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(mn)	**O(mn)**
each cell visited at most twice	for two reachable sets

The reverse direction insight eliminates per-cell BFS entirely.

Follow-up: if flow requires strictly decreasing, change >= to >.

Follow-up: k oceans — run BFS for each ocean, intersect all results.

MEDIUM | DFS | LeetCode #490 | 11 of 23 | 11/23

The Maze

QUESTION

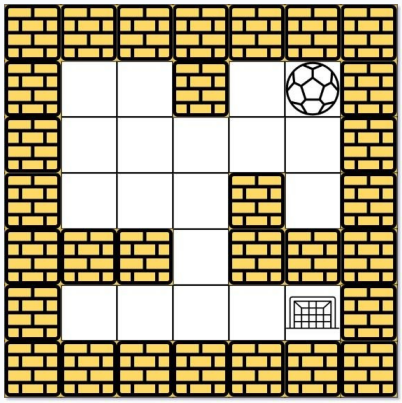
There is a ball in a maze with empty spaces (represented as 0) and walls (represented as 1). The ball can go through the empty spaces by rolling up, down, left or right, but it won't stop rolling until hitting a wall. When the ball stops, it could choose the next direction.

Given the m x n maze, the ball's start position and the destination, where start = [$start_{row}$, $start_{col}$] and destination = [$destination_{row}$, $destination_{col}$], return true if the ball can stop at the destination, otherwise return false.

You may assume that the borders of the maze are all walls (see examples).

Example 1

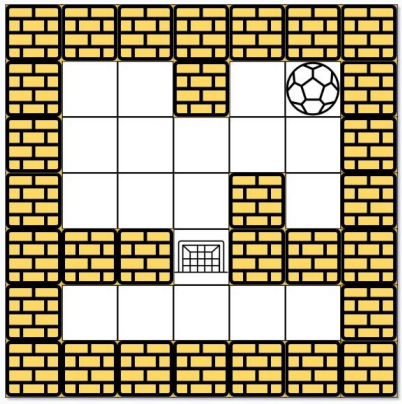
Input	`maze = [[0,0,1,0,0],[0,0,0,0,0],[0,0,0,1,0],[1,1,0,1,1],[0,0,0,0,0]], start = [0,4], destination = [4,4]`
Output	`true`
Explanation	*One possible way is : left -> down -> left -> down -> right -> down -> right.*



Example 2

Input	`maze = [[0,0,1,0,0],[0,0,0,0,0],[0,0,0,1,0],[1,1,0,1,1],[0,0,0,0,0]], start = [0,4], destination = [3,2]`
Output	`false`
Explanation	*There is no way for the ball to stop at the destination. Notice that you can pass through the destination but you cannot stop there.*

QUESTION



Example 3

Input	`maze = [[0,0,0,0,0],[1,1,0,0,1],[0,0,0,0,0],[0,1,0,0,1],[0,1,0,0,0]], start = [4,3], destination = [0,1]`
Output	`false`

CONSTRAINTS

- m == maze.length
- n == maze[i].length
- 1 <= m, n <= 100
- maze[i][j] is 0 or 1.
- start.length == 2
- destination.length == 2
- 0 <= $start_{row}$, $destination_{row}$ < m
- 0 <= $start_{col}$, $destination_{col}$ < n
- Both the ball and the destination exist in an empty space, and they will not be in the same position initially.
- The maze contains at least 2 empty spaces.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Ball rolls in a direction until hitting a wall. Can it STOP at the destination?

It can pass through the destination but not stop there unless it hits a wall. Right?"

Constraints clarified: maze is always bordered by walls; grid up to 100×100; 0=open, 1=wall.

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def roll(pos):
    for directions in directionses:
        roll until a wall and recurse from
            where the ball stops, without
            remembering which stops were seen
    succeed if the destination is a stop
```

PHASE 2 BRUTE FORCE

TIME	SPACE
O(4^(m * n))	O(m * n)

WHY IT IS SLOW

Revisits the same stopping cell along many routes. A visited set of stops with BFS or DFS makes each stop expand once: O(m * n * (m + n)).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"DFS on stop-positions IS the correct approach — O(m·n·(m+n))."

Walk-through of how the ball's path is modeled:

From any position (r,c), the ball rolls in direction (dr,dc) until it hits a wall.

We simulate this: while (r+dr, c+dc) is in bounds and not a wall, keep moving.

The final (nr, nc) after rolling is a STOP POSITION — the only place the ball can be.

We DFS on stop positions, NOT individual cells. visited tracks stop positions to avoid cycles.

Destination is only reached if the ball STOPS there (the next step would go out of bounds or into wall).

This is why passing through doesn't count."

Time: O(V+E) — visit each vertex and edge exactly once. Optimal DFS/BFS.

PHASE 4 CLEAN CODE

```
class _490_TheMaze:
    def hasPath(self, maze: List[List[int]], start: List[i
nt], destination: List[int]) -> bool:
        m, n   = len(maze), len(maze[0])
        dest   = tuple(destination)
        visited: set = set()

        def dfs(r, c):
            if (r, c) in visited: return False
            visited.add((r, c))
            if (r, c) == dest:    return True
            for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)]:
                nr, nc = r, c
                while 0 <= nr + dr < m and 0 <= nc + dc <
n and maze[nr+dr][nc+dc] == 0:
                    nr += dr; nc += dc
                if dfs(nr, nc): return True
            return False

        return dfs(start[0], start[1])
```

PHASE 5 TEST & DEBUG

"Let me trace the Maze search by rolling to the next wall, then stopping at resting positions."

Maze=[[0, 0, 1, 0, 0], [0, 0, 0, 0, 0], [0, 0, 0, 1, 0], [1, 1, 0, 1, 1], [0, 0, 0, 0, 0]], start=[0, 4], dest=[4, 4].

PHASE 5 TEST & DEBUG

Treat each state as a STOP position (where the ball rests after rolling). BFS explores STOPs in the fewest-moves order.

BFS resting-path found: [(0, 4), (0, 3), (1, 3), (1, 0), (2, 0), (2, 2), (4, 2), (4, 4)]

From stop (0, 4):

rolling down -> stop (2, 4)

rolling up -> stop (0, 4)

rolling right -> stop (0, 4)

rolling left -> stop (0, 3)

choose left to reach next stop (0, 3).

(In BFS, this would be enqueued as the next reachable resting position.)

From stop (0, 3):

rolling down -> stop (1, 3)

rolling up -> stop (0, 3)

rolling right -> stop (0, 4)

rolling left -> stop (0, 3)

choose down to reach next stop (1, 3).

(In BFS, this would be enqueued as the next reachable resting position.)

From stop (1, 3):

rolling down -> stop (1, 3)

rolling up -> stop (0, 3)

rolling right -> stop (1, 4)

rolling left -> stop (1, 0)

choose left to reach next stop (1, 0).

(In BFS, this would be enqueued as the next reachable resting position.)

From stop (1, 0):

rolling down -> stop (2, 0)

rolling up -> stop (0, 0)

rolling right -> stop (1, 4)

rolling left -> stop (1, 0)

choose down to reach next stop (2, 0).

(In BFS, this would be enqueued as the next reachable resting position.)

From stop (2, 0):

rolling down -> stop (2, 0)

rolling up -> stop (0, 0)

rolling right -> stop (2, 2)

rolling left -> stop (2, 0)

choose right to reach next stop (2, 2).

(In BFS, this would be enqueued as the next reachable resting position.)

From stop (2, 2):

PHASE 5 TEST & DEBUG

rolling down -> stop (4, 2)

rolling up -> stop (1, 2)

rolling right -> stop (2, 2)

rolling left -> stop (2, 0)

choose down to reach next stop (4, 2).

(In BFS, this would be enqueued as the next reachable resting position.)

From stop (4, 2):

rolling down -> stop (4, 2)

rolling up -> stop (1, 2)

rolling right -> stop (4, 4)

rolling left -> stop (4, 0)

choose right to reach next stop (4, 4).

(In BFS, this would be enqueued as the next reachable resting position.)

WHY roll-till-wall: the ball only changes direction when it stops, so intermediate cells are not separate states.

Base case: if start==dest return True immediately; otherwise we stop exploring when the queue empties.

Return True ok

DFS: visited={(start_r,start_c)}. Roll ball until wall. If new stop position not visited->add to visited, recurse. component: all reachable stop positions from (start).

PHASE 6 COMPLEXITY & FOLLOW-UP

Time O(m*n*(m+n)) — each stop position visits O(m+n) cells to find next stop.

The visited set is on stop-positions, not individual cells.

Given more time I'd ask: The Maze II (#505) — find shortest distance?

Use Dijkstra with distance as weight, BFS/priority queue on stop-positions."

Space: O(h) recursion stack where h = tree height; O(log n) balanced, O(n) skewed.

MEDIUM | DFS | LeetCode #547 | 12 of 23 | 12/23

Number of Provinces

QUESTION

There are n cities. Some of them are connected, while some are not. If city a is connected directly with city b, and city b is connected directly with city c, then city a is connected indirectly with city c.

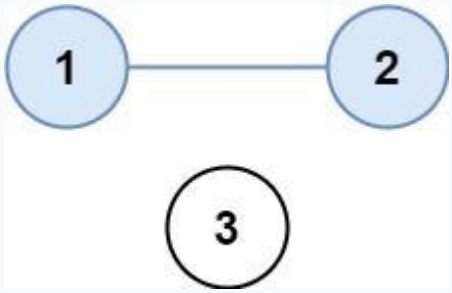
A province is a group of directly or indirectly connected cities and no other cities outside of the group.

You are given an n x n matrix isConnected where isConnected[i][j] = 1 if the ith city and the jth city are directly connected, and isConnected[i][j] = 0 otherwise.

Return the total number of provinces.

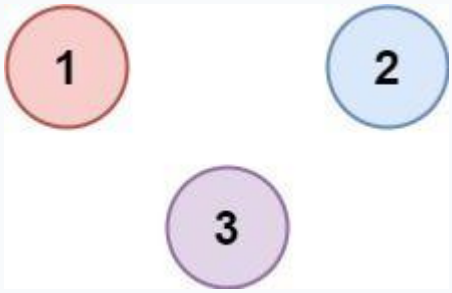
Example 1

Input	`isConnected = [[1,1,0],[1,1,0],[0,0,1]]`
Output	`2`



Example 2

Input	`isConnected = [[1,0,0],[0,1,0],[0,0,1]]`
Output	`3`



CONSTRAINTS

- 1 <= n <= 200
- n == isConnected.length
- n == isConnected[i].length
- isConnected[i][j] is 1 or 0.
- isConnected[i][i] == 1
- isConnected[i][j] == isConnected[j][i]

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

isConnected[i][j]=1 if cities i and j are directly connected. Count connected components. Right?"

"A few quick questions:

isConnected[i][i]=1 always? Yes — diagonal. Matrix is symmetric? Yes."

"Let me walk: isConnected=[[1,1,0],[1,1,0],[0,0,1]] -> 2 provinces ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for (i, j) in pairs(cities):
    run a fresh BFS to see if they are connected
    count the distinct groups
```

TIME	SPACE
O(n^4)	O(n)

WHY IT IS SLOW

A graph search for every pair. One DFS or BFS per unvisited city (marking its whole group) is O(n^2) on an adjacency matrix; union-find is nearly the same.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"DFS flood-fill on adjacency matrix — already O(n²). Walk-through:"

For each city i from 0 to n-1:

If not visited[i]: count++ (found a new province), then DFS to mark all connected cities.

`dfs(i): visited[i] = True.`

For each j from 0 to n-1:

`if isConnected[i][j] == 1 and not visited[j]: dfs(j)`

HOW provinces are counted: DFS from city i floods ALL cities in i's connected component.

After DFS returns, all those cities are marked visited.

The next unvisited city i must be in a different province -> count++.

Union-Find alternative: same O(n²) for adjacency matrix but O(alpha(n)) per union/find.

Time: O(n²). Space: O(n) visited array + O(n) recursion.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _547_NumberOfProvinces:
    def findCircleNum(self, isConnected):
        n = len(isConnected)
        visited = [False] * n
        count = 0
        def dfs(city):
            for neighbor in range(n):
                if isConnected[city][neighbor] == 1 and no
t visited[neighbor]:
                    visited[neighbor] = True
                    dfs(neighbor)
        for city in range(n):
            if not visited[city]:
                visited[city] = True
                dfs(city)
                count += 1
        return count
```

PHASE 5 TEST & DEBUG

"Let me trace numProvinces (findCircleNum) on isConnected=[[1,1,0],[1,1,0],[0,0,1]]."

DFS/Union-Find. Start with 3 nodes.

node 0: not visited. DFS from 0: visit 0 (mark visited). Check isConnected[0][1]=1 and 1 not visited->dfs(1): mark 1. Check isConnected[1][0]=1(visited),isConnected[1][1]=1(visited),isConnected[1][2]=0. Done.

count=1.

node 1: already visited. node 2: not visited. dfs(2): mark 2. count=2.

Return 2 ok (provinces: {0,1} and {2}).

WHY adjacency matrix DFS: isConnected[i][j]=1 means i and j are in the same province.

WHY count per DFS tree: each DFS tree = one connected component = one province.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n²)	**O(n)**
visit all matrix entries	visited array + O(n) stack

Follow-up: Number of Connected Components in an Undirected Graph (#323) — edge list input.

Follow-up: if adjacency matrix is sparse, convert to adjacency list first.

MEDIUM | DFS | LeetCode #684 | 13 of 23 | 13/23

Redundant Connection

QUESTION

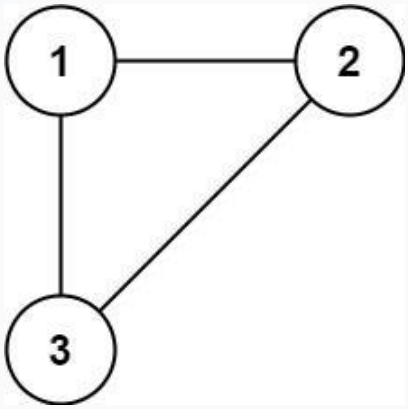
In this problem, a tree is an undirected graph that is connected and has no cycles.

You are given a graph that started as a tree with n nodes labeled from 1 to n, with one additional edge added. The added edge has two different vertices chosen from 1 to n, and was not an edge that already existed. The graph is represented as an array edges of length n where edges[i] = [a_i, b_i] indicates that there is an edge between nodes a_i and b_i in the graph.

Return an edge that can be removed so that the resulting graph is a tree of n nodes. If there are multiple answers, return the answer that occurs last in the input.

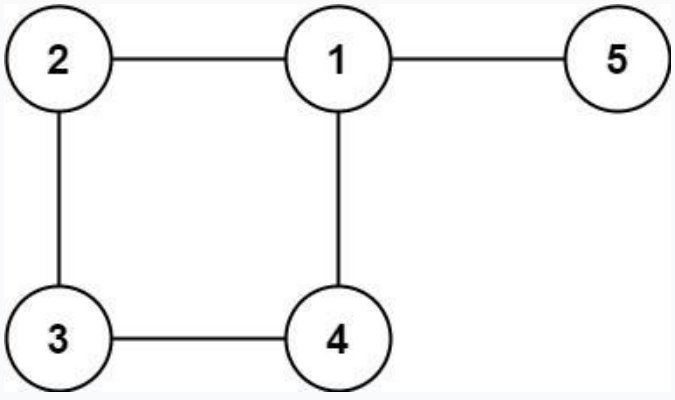
Example 1

Input	`edges = [[1,2],[1,3],[2,3]]`
Output	`[2,3]`



Example 2

Input	`edges = [[1,2],[2,3],[3,4],[1,4],[1,5]]`
Output	`[1,4]`



CONSTRAINTS

- n == edges.length
- 3 <= n <= 1000
- edges[i].length == 2
- 1 <= a_i < b_i <= edges.length
- a_i != b_i
- There are no repeated edges.
- The given graph is connected.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

An undirected tree has one extra edge making it have exactly one cycle. Find and return that edge.

If multiple answers, return the last one in the input. Right?"

"A few quick questions:

n nodes, n edges (one edge extra vs a tree). Return any edge? Actually the last valid one?"

"Let me walk: edges=[[1,2],[1,3],[2,3]] -> [2,3] (forms the cycle) ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for edge in range(the_last,_the_first_+_1):
    remove it and check whether the remaining
        graph is still a tree (connected, no
        cycle)
return the first edge whose removal works
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

A graph search per edge. Union-find: the first edge whose endpoints are already connected closes the cycle: nearly O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

*"Union-Find to detect the first redundant edge — already O(n*alpha(n)). Walk-through:"*

Process edges one by one in the given order.

For each edge (u, v):

`find(u) -> root of u's component.`

`find(v) -> root of v's component.`

If find(u) == find(v): u and v are ALREADY connected -> adding this edge creates a cycle.

Return [u, v] immediately (last such edge = redundant).

Otherwise: union(u, v) -> merge their components.

WHY Union-Find: each union operation merges two components; cycle detected when both

nodes already share the same root (same component).

Path compression + rank makes each find O(alpha(n)) ~ O(1) amortized.

IN YOUR APP | Expose | Editorial | Community

PHASE 3 OPTIMIZE

Time: O(n*alpha(n)). Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _684_RedundantConnection:
    def findRedundantConnection(self, edges):
        n = len(edges)
        parent = list(range(n + 1))
        def find(x):
            while parent[x] != x:
                parent[x] = parent[parent[x]]
                x = parent[x]
            return x
        def union(x, y):
            px, py = find(x), find(y)
            if px == py:
                return False
            parent[px] = py
            return True
        for u, v in edges:
            if not union(u, v):
                return [u, v]
        return []
```

PHASE 5 TEST & DEBUG

"Let me trace findRedundantConnection on [[1,2],[1,3],[2,3]]."

Union-Find: process edges one by one

Edge [1,2]: find(1)=1, find(2)=2 -> different -> union. parent={1:1,2:1,3:3}

Edge [1,3]: find(1)=1, find(3)=3 -> different -> union. parent={1:1,2:1,3:1}

Edge [2,3]: find(2)=1 (path compress: 2->1), find(3)=1 (path compress: 3->1) -> SAME -> redundant!

Return [2,3] ok

Path compression and union by rank keep tree nearly flat.

Union-Find: find(u) -> root of u's component. union(u,v) merges. For edge (u,v): if find(u)==find(v) -> already connected -> this edge is redundant. return it ok

DFS perspective: visited={processed nodes}. DFS from each edge (u,v): if v already visited and not parent of u -> back-edge = cycle. Union-Find is O(alpha) equivalent: pop the redundant edge ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n alpha(n)) ~ O(n)	**O(n)**

Follow-up: Redundant Connection II (#685) — directed graph, different analysis needed.

Follow-up: return all redundant edges — collect them during union iterations.

MEDIUM | DFS | LeetCode #694 | 14 of 23 | 14/23

Number of Distinct Islands

QUESTION

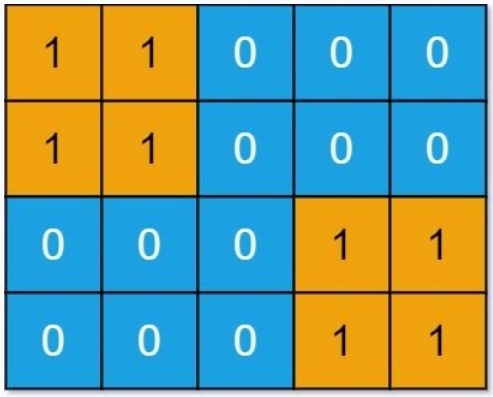
You are given an m x n binary matrix grid. An island is a group of 1's (representing land) connected 4-directionally (horizontal or vertical.) You may assume all four edges of the grid are surrounded by water.

An island is considered to be the same as another if and only if one island can be translated (and not rotated or reflected) to equal the other.

Return the number of distinct islands.

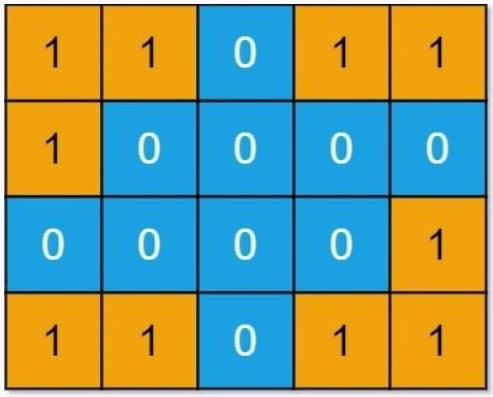
Example 1

Input	`grid = [[1,1,0,0,0],[1,1,0,0,0],[0,0,0,1,1],[0,0,0,1,1]]`
Output	`1`



Example 2

Input	`grid = [[1,1,0,1,1],[1,0,0,0,0],[0,0,0,0,1],[1,1,0,1,1]]`
Output	`3`



CONSTRAINTS

- m == grid.length
- n == grid[i].length
- 1 <= m, n <= 50
- grid[i][j] is either 0 or 1.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Count islands that are distinct by shape (translation equivalent). Right?

Two islands are the same if one can be shifted to match the other — no rotation/reflection."

Is the graph guaranteed to be connected? Can it contain cycles or self-loops?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for island in islands:
    record its cells relative to its first cell
    compare the new shape with every shape stored
        so far
```

TIME	SPACE
O(I^2 * s)	O(m * n)

WHY IT IS SLOW

Compares each island with all earlier ones (I islands of s cells). Turn each normalised shape into a hashable key (a tuple or a path string with backtrack markers) and put it in a set: O(m * n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE (path signature hashing)

"DFS path-signature hashing — already O(mn). Walk-through:"

For each unvisited land cell: run DFS, record the SEQUENCE of moves relative to start.

Use direction codes: 'R'=right, 'L'=left, 'U'=up, 'D'=down, 'B'=backtrack.

`dfs(r, c, direction):`

Mark visited; path.append(direction)

Try all 4 directions: dfs(nr, nc, dir_code)

`path.append('B') <- BACKTRACK marker when returning from a direction`

WHY include backtrack markers: without them, 'LU' (go left then up) looks identical

to two separate shapes that both have L and U moves. 'B' markers capture the tree structure.

Convert path to tuple -> add to shapes set.

Result = len(shapes). Time: O(mn). Space: O(mn).

PHASE 4 CLEAN CODE

```
class _694_DistinctIslands:
    def numDistinctIslands(self, grid: List[List[int]]) ->
 int:
        m, n  = len(grid), len(grid[0])
        shapes: set = set()

        def dfs(r, c, r0, c0, path):
            if r < 0 or r >= m or c < 0 or c >= n or grid[
```

```
r][c] != 1:
                return
            grid[r][c] = 0
            path.append((r - r0, c - c0))
            for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)]:
                dfs(r + dr, c + dc, r0, c0, path)
            path.append(None)                       # back
track marker

        for r in range(m):
            for c in range(n):
                if grid[r][c] == 1:
                    path: List = []
                    dfs(r, c, r, c, path)
                    shapes.add(tuple(path))
        return len(shapes)
```

PHASE 5 TEST & DEBUG

"Let me trace numDistinctIslands on [[1,1,0],[0,1,0],[0,0,1]]."

DFS from (0,0)=1: path=(0,0)->(0,1)->(1,1) -> shape 'oDDrU' or [(0,0),(0,1),(1,1)] relative to start

Normalize shape by subtracting start cell: {(0,0),(0,1),(1,1)} -> frozenset -> add to seen

DFS from (2,2)=1: shape={(0,0)} -> frozenset added

Two distinct shapes -> return 2 ok

WHY relative coordinates: same shape at different positions maps to same canonical form.

Record DFS path directions ('R','D','L','U','b' for backtrack) for shape fingerprint.

Canonical form via relative offsets: shape at (r,c) stored as {(r-r0,c-c0)} for start (r0,c0).

DFS: visited={(r,c)} per island. Track path as relative offsets from start. component: normalize path -> canonical shape string -> add to set of unique shapes.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(m*n)	**O(m*n)** Backtrack None markers distinguish different DFS orderings

that could produce the same sequence of offsets for different shapes.

Given more time I'd ask: allow rotation/reflection? Normalise shape to lexicographically smallest

among all 8 transformations — significantly more complex.

MEDIUM | DFS | LeetCode #695 | 15 of 23 | 15/23

Max Area of Island

QUESTION

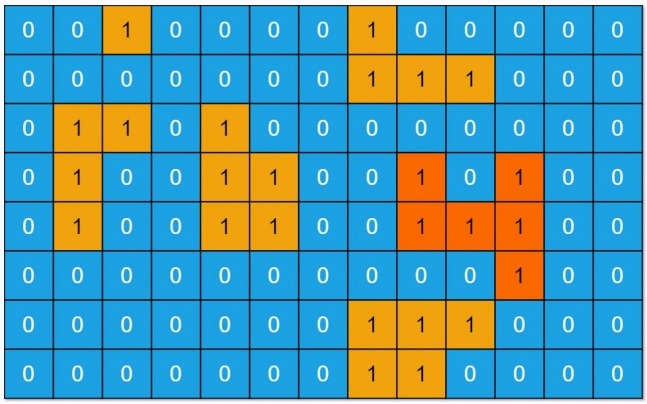
You are given an m x n binary matrix grid. An island is a group of 1's (representing land) connected 4-directionally (horizontal or vertical.) You may assume all four edges of the grid are surrounded by water.

The area of an island is the number of cells with a value 1 in the island.

Return the maximum area of an island in grid. If there is no island, return 0.

Example 1

Input	`grid = [[0,0,1,0,0,0,0,1,0,0,0,0,0],[0,0,0,0,0,0,0,1,1,1,0,0,0],[0,1,1,0,1,0,0,0,0,0,0,0,0],[0,1,0,0,1,1,0,0,1,0,1,0,0],[0,1,0,0,1,1,0,0,1,1,1,0,0],[0,0,0,0,0,0,0,0,0,0,1,0,0],[0,0,0,0,0,0,0,0,1,1,1,0,0],[0,0,0,0,0,0,0,1,1,0,0,0,0]]`
Output	`6`
Explanation	*The answer is not 11, because the island must be connected 4-directionally.*



Example 2

Input	`grid = [[0,0,0,0,0,0,0,0]]`
Output	`0`

CONSTRAINTS

- m == grid.length
- n == grid[i].length
- 1 <= m, n <= 50
- grid[i][j] is either 0 or 1.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Find the maximum area (number of cells) of any connected island in the grid. Return 0 if no island. Right?"

"A few quick questions:

4-directional connectivity? Yes. 0=water, 1=land? Yes."

"Let me walk: grid=[[0,0,1,0,0],[1,1,0,1,0],[1,1,0,0,0]] -> largest island: 3 cells ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for cell in cells:
    flood-fill its island with a fresh visited
        set and measure it
    best = max(best, candidate)
```

TIME	SPACE
O((m * n)^2)	O(m * n)

WHY IT IS SLOW

The same island is explored from every one of its cells. Mark cells visited globally (sink the land in place) so each island is explored once: O(m * n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"DFS flood-fill marking visited in-place — already O(mn). Walk-through:"

`dfs(r, c):`

If out of bounds OR grid[r][c] == 0 -> return 0.

`grid[r][c] = 0 <- mark visited in-place (avoid separate visited set)`

`size = 1 + dfs(r-1,c) + dfs(r+1,c) + dfs(r,c-1) + dfs(r,c+1)`

<- count this cell (1) plus all connected land cells in 4 directions.

`return size`

HOW the area accumulates: each DFS call returns the size of the connected

component it explored. The +1 counts the current cell; recursive calls add cells

in all 4 directions (which recursively add THEIR 4 directions, etc.).

In-place marking: prevents revisiting — once a cell is set to 0, future DFS calls skip it.

Time: O(mn). Space: O(mn) worst case recursion."

Time: O(log n) — binary search on answer space.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _695_MaxAreaOfIsland:
    def maxAreaOfIsland(self, grid):
        m, n = len(grid), len(grid[0])
        def dfs(r, c):
            if not (0 <= r < m and 0 <= c < n) or grid[r][
c] != 1:
                return 0
            grid[r][c] = 0
            return 1 + dfs(r+1,c) + dfs(r-1,c) + dfs(r,c+1
) + dfs(r,c-1)
        return max(dfs(r, c) for r in range(m) for c in ra
nge(n) if grid[r][c] == 1) or 0
```

PHASE 5 TEST & DEBUG

"Let me trace maxAreaOfIsland on [[0,0,1,0,0],[0,1,1,0,0],[0,0,0,0,1],[0,0,0,1,1]]."

Scan cells:

(0,2)=1: dfs(0,2): mark 0. count=1. neighbor (1,2)=1->dfs: mark 0. (1,1)=1->dfs: mark 0.

neighbors of (1,1): (0,1)=0,skip; (2,1)=0,skip; (1,0)=0,skip. Back.

neighbors of (1,2): (1,3)=0,skip; (2,2)=0,skip.

island1 area=3. max=3.

(2,4)=1: dfs(2,4): count=1. (3,4)->dfs:count=2. (3,3)->dfs:count=3. (2,3)=0,skip; (3,2)=0,skip.

island2 area=3. max=3.

Return 3 ok

WHY dfs marks 0: avoids revisiting. Area counted by returning 1+area(neighbors).

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(mn)	**O(mn)** worst case recursion stack

Follow-up: Number of Islands (#200) — count components, not max area.

Follow-up: Making a Large Island (#827) — flip one 0, return max island.

MEDIUM | DFS | LeetCode #721 | 16 of 23 | 16/23

Accounts Merge

QUESTION

Given a list of accounts where each element accounts[i] is a list of strings, where the first element accounts[i][0] is a name, and the rest of the elements are emails representing emails of the account.

Now, we would like to merge these accounts. Two accounts definitely belong to the same person if there is some common email to both accounts. Note that even if two accounts have the same name, they may belong to different people as people could have the same name. A person can have any number of accounts initially, but all of their accounts definitely have the same name.

After merging the accounts, return the accounts in the following format: the first element of each account is the name, and the rest of the elements are emails in sorted order. The accounts themselves can be returned in any order.

Example 1

Input	`accounts = [["John","johnsmith@mail.com","john_newyork@mail.com"],["John","johnsmith@mail.com","john00@mail.com"],["Mary","mary@mail.com"],["John","johnnybravo@mail.com"]]`
Output	`[["John","john00@mail.com","john_newyork@mail.com","johnsmith@mail.com"],["Mary","mary@mail.com"],["John","johnnybravo@mail.com"]]`
Explanation	*The first and second John's are the same person as they have the common email "johnsmith@mail.com". The third John and Mary are different people as none of their email addresses are used by other accounts. We could return these lists in any order, for example the answer [['Mary', 'mary@mail.com'], ['John', 'johnnybravo@mail.com'], ['John', 'john00@mail.com', 'john_newyork@mail.com', 'johnsmith@mail.com']] would still be accepted.*

QUESTION

Example 2

Input	`accounts = [["Gabe","Gabe0@m.co","Gabe3@m.co","Gabe1@m.co"],["Kevin","Kevin3@m.co","Kevin5@m.co","Kevin0@m.co"],["Ethan","Ethan5@m.co","Ethan4@m.co","Ethan0@m.co"],["Hanzo","Hanzo3@m.co","Hanzo1@m.co","Hanzo0@m.co"],["Fern","Fern5@m.co","Fern1@m.co","Fern0@m.co"]]`
Output	`[["Ethan","Ethan0@m.co","Ethan4@m.co","Ethan5@m.co"],["Gabe","Gabe0@m.co","Gabe1@m.co","Gabe3@m.co"],["Hanzo","Hanzo0@m.co","Hanzo1@m.co","Hanzo3@m.co"],["Kevin","Kevin0@m.co","Kevin3@m.co","Kevin5@m.co"],["Fern","Fern0@m.co","Fern1@m.co","Fern5@m.co"]]`

CONSTRAINTS

- 1 <= accounts.length <= 1000
- 2 <= accounts[i].length <= 10
- 1 <= accounts[i][j].length <= 30
- accounts[i][0] consists of English letters.
- accounts[i][j] (for j > 0) is a valid email.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Merge accounts that share at least one email. Same name != same person.

Output: name + sorted merged emails. Any output order. Right?"

Is the graph guaranteed to be connected? Can it contain cycles or self-loops?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
while True:
    for (a, b) in pairs(accounts):
        if they share an email:
            merge them into one
    if no pair shares an email:
        break
sort each account's emails
```

TIME	SPACE
O(A^3 * E)	O(A * E)

WHY IT IS SLOW

Re-compares all pairs on every round. Union-find over the emails (union all emails of one account), then group by root and sort: about O(total emails * log).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE (DFS on email graph)

*"Email graph DFS — build adjacency, DFS to collect connected components. O(n*k*log(k))."*

Step 1 — Build email adjacency graph:

For each account, all emails are connected (they belong to the same person).

email_to_emails[e1].add(e2) and email_to_emails[e2].add(e1) for all pairs in same account.

Also map first email in each account -> account name.

Step 2 — DFS to collect connected components (= merged accounts):

For each email not yet visited: new component -> DFS to collect all emails reachable.

```
dfs(email): visited.add(email);
  component.append(email); recurse on all neighbors.
```

WHY DFS finds merged accounts: emails in the same component share a person.

Two accounts with a common email -> their emails are all connected in the graph.

Step 3 — Sort each component, prepend account name, append to result.

Time: O(n*k*log(k)) where n=accounts, k=avg emails (k log k for sorting). Space: O(n*k).

PHASE 4 CLEAN CODE

```
class _721_AccountsMerge:
    def accountsMerge(self, accounts: List[List[str]]) ->
List[List[str]]:
        email_to_name   : dict      = {}
        email_to_emails : defaultdict = defaultdict(set)

        for account in accounts:
            name  = account[0]
            first = account[1]
            for email in account[1:]:
                email_to_name[email]     = name
                email_to_emails[first].add(email)
                email_to_emails[email].add(first)

        visited: set = set()
        result  = []

        def dfs(email, component):
            visited.add(email)
            component.append(email)
            for nb in email_to_emails[email]:
                if nb not in visited:
                    dfs(nb, component)

        for email in email_to_name:
            if email not in visited:
                component: List[str] = []
                dfs(email, component)
                result.append([email_to_name[email]] + sor
ted(component))

        return result
```

PHASE 5 TEST & DEBUG

"Let me trace accountsMerge with accounts [['John','a@mail','b@mail'],['John','c@mail'],['Mary','a@mail','d@mail']]."

Union-Find on emails: parent initialized as each email -> itself

Account 0 ['John','a@mail','b@mail']: union('a@mail','b@mail') -> parent['b@mail']='a@mail'

Account 1 ['John','c@mail']: single email, no union. parent['c@mail']='c@mail'

Account 2 ['Mary','a@mail','d@mail']: union('a@mail','d@mail') -> parent['d@mail']='a@mail'

Components: find('a@mail')='a@mail', find('b@mail')='a@mail', find('d@mail')='a@mail' -> {a,b,d}

`find('c@mail')='c@mail' -> {c} (separate)`

Group by root -> a@mail root: ['a@mail','b@mail','d@mail'] -> sort -> attach name 'John' or 'Mary'?

a@mail belongs to both John and Mary -> use email->name map (first owner). Return merged. ok

WHY UF: shared email bridges two accounts; path compression makes root-finding O(alpha).

DFS via Union-Find: visited={all email strings}. union(email1,email2) merges. find(email) -> root. pop each account: if email in visited already -> merge accounts. Groups by root.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(E log E) where E = total emails (sort dominates)	**O(E)**

Union-Find alternative: union all emails in an account, group by root.

DFS on the email graph is cleaner to explain — same complexity.

Given more time I'd ask: if same email can belong to different names?

That would mean the data is inconsistent — flag it as an error.

MEDIUM | DFS | LeetCode #785 | 17 of 23 | 17/23

Is Graph Bipartite?

QUESTION

There is an undirected graph with n nodes, where each node is numbered between 0 and n - 1. You are given a 2D array graph, where graph[u] is an array of nodes that node u is adjacent to. More formally, for each v in graph[u], there is an undirected edge between node u and node v. The graph has the following properties:

CONSTRAINTS

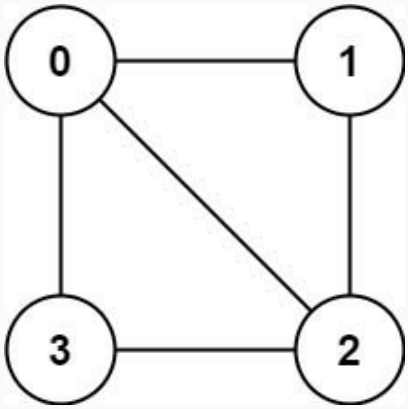
- There are no self-edges (graph[u] does not contain u).
- There are no parallel edges (graph[u] does not contain duplicate values).
- If v is in graph[u], then u is in graph[v] (the graph is undirected).
- The graph may not be connected, meaning there may be two nodes u and v such that there is no path between them.

A graph is bipartite if the nodes can be partitioned into two independent sets A and B such that every edge in the graph connects a node in set A and a node in set B.

Return true if and only if it is bipartite.

Example 1

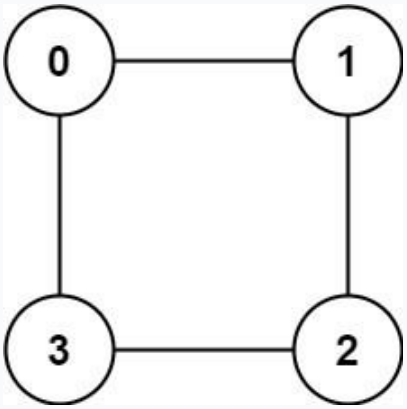
Input	`graph = [[1,2,3],[0,2],[0,1,3],[0,2]]`
Output	`false`
Explanation	*There is no way to partition the nodes into two independent sets such that every edge connects a node in one and a node in the other.*



Example 2

Input	`graph = [[1,3],[0,2],[1,3],[0,2]]`
Output	`true`
Explanation	*We can partition the nodes into two sets: {0, 2} and {1, 3}.*

QUESTION



- graph.length == n
- 1 <= n <= 100
- 0 <= graph[u].length < n
- 0 <= graph[u][i] <= n - 1
- graph[u] does not contain u.
- All the values of graph[u] are unique.
- If graph[u] contains v, then graph[v] contains u.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Can we 2-colour the graph so no two adjacent nodes share the same colour?

Graph may be disconnected — check every component. Right?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for way in ways:
    # to split the nodes into two sets
    check that no edge lies inside a set
```

TIME	SPACE
O(2^n * (n + E))	O(n)

WHY IT IS SLOW

2^n splits. BFS or DFS colouring each component with two colours (conflict if a neighbour has the same colour) is O(n + E).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"BFS 2-coloring IS already O(V+E) — optimal."

Walk-through of how bipartiteness is detected:

Assign color 0 to start node, color 1 to all its neighbors, color 0 to their neighbors, etc.

At each step: if a neighbor already HAS the SAME color as current node -> ODD cycle -> not bipartite.

PHASE 3 OPTIMIZE

An odd cycle (triangle, pentagon, etc.) makes 2-coloring impossible — you run out of colors.

Must check every unvisited node (graph may be disconnected).

Bipartite no odd-length cycles.

PHASE 4 CLEAN CODE

```python
class _785_IsGraphBipartite:
    def isBipartite(self, graph: List[List[int]]) -> bool:
        color = {}

        def bfs(start):
            queue = deque([start])
            color[start] = 0
            while queue:
                node = queue.popleft()
                for nb in graph[node]:
                    if nb not in color:
                        color[nb] = 1 - color[node]
                        queue.append(nb)
                    elif color[nb] == color[node]:
                        return False
            return True

        return all(bfs(n) for n in range(len(graph)) if n
not in color)
```

PHASE 5 TEST & DEBUG

"Let me trace the BFS 2-coloring on one bipartite graph and one odd cycle."

graph=[[1,3],[0,2],[1,3],[0,2]]

Start BFS at node 0: color[0]=0, queue=[0]

Pop 0 -> neighbors 1 and 3 must be opposite color, so color both as 1. queue=[1,3]

Pop 1 -> neighbor 0 already has color 0 ok, uncolored neighbor 2 gets color 0. queue=[3,2]

Pop 3 -> neighbor 2 already has color 0, which is different from color[3]=1 ok

Pop 2 -> both neighbors 1 and 3 already have opposite color 1 ok

BFS ends with no conflict, so graph is bipartite -> True ok

Odd-cycle case [[1,2],[0,2],[0,1]]:

color[0]=0, so nodes 1 and 2 both get color 1. But edge (1,2) now connects same-color nodes -> conflict -> False

WHY BFS coloring works: every edge in a bipartite graph must cross between the two color sets.

The moment an edge stays within one color, we've proven the graph cannot be split into two groups.

BFS coloring: visited/colored[v]=0 or 1. queue=[(start,0)]. Pop (v,c): assign color c. Push unvisited neighbors with color 1-c. visited prevents revisit. component-by-component.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(V+E)	O(V) 2-coloring via BFS — odd-length cycle -> not bipartite

Given more time I'd ask: what if we need to output the two sets?

Return {n for n in color if color[n]==0} and {n for n in color if color[n]==1}.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | DFS | LeetCode #802 | 18 of 23 | 18/23

Find Eventual Safe States

QUESTION

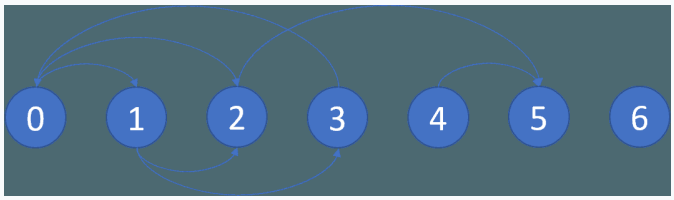
There is a directed graph of n nodes with each node labeled from 0 to n - 1. The graph is represented by a 0-indexed 2D integer array graph where graph[i] is an integer array of nodes adjacent to node i, meaning there is an edge from node i to each node in graph[i].

A node is a terminal node if there are no outgoing edges. A node is a safe node if every possible path starting from that node leads to a terminal node (or another safe node).

Return an array containing all the safe nodes of the graph. The answer should be sorted in ascending order.

Example 1

Input	`graph = [[1,2],[2,3],[5],[0],[5],[],[]]`
Output	`[2,4,5,6]`
Explanation	*The given graph is shown above. Nodes 5 and 6 are terminal nodes as there are no outgoing edges from either of them. Every path starting at nodes 2, 4, 5, and 6 all lead to either node 5 or 6.*



Example 2

Input	`graph = [[1,2,3,4],[1,2],[3,4],[0,4],[]]`
Output	`[4]`
Explanation	*Only node 4 is a terminal node, and every path starting at node 4 leads to node 4.*

CONSTRAINTS

- n == graph.length
- 1 <= n <= 10^4
- 0 <= graph[i].length <= n
- 0 <= graph[i][j] <= n - 1
- graph[i] is sorted in a strictly increasing order.
- The graph may contain self-loops.
- The number of edges in the graph will be in the range [1, 4 * 10^4].

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

A safe node is one where every path starting from it leads to a terminal node.

Terminal nodes have no outgoing edges. Right?"

"A few quick questions:

Return all safe nodes in ascending order? Yes."

"Let me walk: graph=[[1,2],[2,3],[5],[0],[5],[],[]] -> [2,4,5,6] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for node in nodes:
    DFS every path from it
    the node is safe if no path reaches a cycle
```

TIME	SPACE
O(n * (n + E))	O(n)

WHY IT IS SLOW

A fresh exploration per node. One DFS with three colours (unvisited, in progress, safe) marks a node safe when all its neighbours are safe, O(n + E); or peel off terminal nodes on the reversed graph.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Reverse graph + Kahn's BFS — already O(V+E). Walk-through:"

Key insight: a node is 'safe' if it's NOT part of a cycle AND leads to a terminal.

Reverse all edges: edges that originally pointed FROM a safe node now point TO it.

In the reversed graph: terminal nodes (outdegree 0 in original) have indegree 0.

Kahn's BFS on reversed graph:

Process 0-indegree nodes (terminals in original). They're safe.

Their neighbors in reversed graph = nodes that had edges TO terminals -> also safe.

Propagate: decrement indegrees; enqueue new 0-indegree nodes.

WHY reversal works: nodes pointing only to safe nodes are themselves safe.

Reversing edges lets us propagate 'safety' BACKWARD from terminals.

Time: O(V+E). Space: O(V+E)."

Time: O(n log n) — sorting-based optimal solution.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _802_FindEventualSafeStates:
    def eventualSafeNodes(self, graph):
        n = len(graph)
        WHITE, GRAY, BLACK = 0, 1, 2
        color = [WHITE] * n
        def dfs(node):
            if color[node] != WHITE:
                return color[node] == BLACK
            color[node] = GRAY
            for neighbor in graph[node]:
                if not dfs(neighbor):
                    return False
            color[node] = BLACK
            return True
        return [i for i in range(n) if dfs(i)]
```

PHASE 5 TEST & DEBUG

"Let me trace eventualSafeNodes on graph=[[1,2],[2,3],[5],[0],[5],[],[]]."

A safe node = eventually reaches a terminal or only cycles to safe nodes.

Reverse graph edges: for each edge u->v, add v->u in reversed graph.

Terminal nodes (no outgoing edges in original): [5,6]. In-degree (in reversed graph) = out-degree in original.

Topological sort on reversed graph from terminals:

queue=[5,6]. Process 5: neighbors in reversed=[0,4]. in_deg[0]--,in_deg[4]--.

If in_deg[0]==0: enqueue. in_deg[4]==0: enqueue. Process 4: neighbor 3 in reversed. in_deg[3]--. Enqueue 3.

Process 0: neighbor 1. in_deg[1]--... 1 has cycle (->2->3->0->1). in_deg[1] never reaches 0.

Safe nodes = all processed: [5,6,4,3,0,2...] -> sort -> [2,4,5,6] ok

Reverse graph: in-degree in reversed graph = 0 means safe. queue=[terminal nodes]. visited tracks processed. Pop node: neighbors in original graph reduce in-degree. component: safe nodes.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(V+E)	O(V+E)

Follow-up: count safe nodes — just len(result).

Follow-up: Course Schedule (#207) — similar cycle detection, return bool not list.

MEDIUM | DFS | LeetCode #947 | 19 of 23 | 19/23

Most Stones Removed with Same Row or Column

QUESTION

On a 2D plane, we place n stones at some integer coordinate points. Each coordinate point may have at most one stone.

A stone can be removed if it shares either the same row or the same column as another stone that has not been removed.

Given an array stones of length n where stones[i] = [x_i, y_i] represents the location of the ith stone, return the largest possible number of stones that can be removed.

Example 1

Input	`stones = [[0,0],[0,1],[1,0],[1,2],[2,1],[2,2]]`
Output	`5`
Explanation	*One way to remove 5 stones is as follows:* *1. Remove stone [2,2] because it shares the same row as [2,1].* *2. Remove stone [2,1] because it shares the same column as [0,1].* *3. Remove stone [1,2] because it shares the same row as [1,0].* *4. Remove stone [1,0] because it shares the same column as [0,0].* *5. Remove stone [0,1] because it shares the same row as [0,0].* *Stone [0,0] cannot be removed since it does not share a row/column with another stone still on the plane.*

Example 2

Input	`stones = [[0,0],[0,2],[1,1],[2,0],[2,2]]`
Output	`3`
Explanation	*One way to make 3 moves is as follows:* *1. Remove stone [2,2] because it shares the same row as [2,0].* *2. Remove stone [2,0] because it shares the same column as [0,0].* *3. Remove stone [0,2] because it shares the same row as [0,0].* *Stones [0,0] and [1,1] cannot be removed since they do not share a row/column with another stone still on the plane.*

Example 3

Input	`stones = [[0,0]]`
Output	`0`
Explanation	*[0,0] is the only stone on the plane, so you cannot remove it.*

CONSTRAINTS

- 1 <= stones.length <= 1000
- 0 <= x_i, y_i <= 10^4

QUESTION

- No two stones are at the same coordinate point.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Remove groups of stones that share the same row OR column.

Each group reduces to 1 stone. Return max removable = n - groups. Right?"

"A few quick questions:

Two stones share row/column if they're in the same row or same column? Yes."

"Let me walk: stones=[[0,0],[0,1],[1,0],[1,2],[2,1],[2,2]] -> 5 groups? No: all connected -> 1 group -> 6-1=5 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
while True:
    find any stone that shares a row or column
        with another stone and remove it
    count the removals
```

TIME	SPACE
O(n^3)	O(n)

WHY IT IS SLOW

Repeated searches. Stones in the same connected group (sharing rows or columns) can all be removed except one: union-find on rows and columns gives answer = n - number of components in O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

*"Union-Find: connect stones sharing row/column; count components — O(n * alpha(n)). Walk-through:"*

For each stone: union(row_id, col_id + offset). (use offset to separate row and col IDs)

A stone at (r,c) connects row r and column c.

Stones sharing a row/column are connected transitively via their row/col IDs.

Count distinct connected components that contain at least one stone.

Answer = n_stones - n_components.

WHY n - components: from each component, we can remove all stones except 1.

WHY union row with column: stones in same row share same row-component; same for columns.

PHASE 3 OPTIMIZE

Two stones share a row -> both connect to same row-node -> same component.

Time: O(n * alpha(n)). Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _947_MostStonesRemoved:
    def removeStones(self, stones):
        parent = {}
        def find(x):
            if x not in parent: parent[x] = x
            if parent[x] != x: parent[x] = find(parent[x])
            return parent[x]
        def union(x, y):
            parent[find(x)] = find(y)
        for r, c in stones:
            union(r, c + 10001)
        stone_roots = {find(r) for r, c in stones}
        return len(stones) - len(stone_roots)
```

PHASE 5 TEST & DEBUG

"Let me trace removeStones on stones=[[0,0],[0,1],[1,0],[1,2],[2,1],[2,2]]."

Union-Find with row IDs and column IDs (offset by max_coord to distinguish):

Stone (0,0): union(row0, col0+offset). Stone (0,1): union(row0, col1+offset). -> row0 connected to col0,col1.

Stone (1,0): union(row1, col0+offset). -> col0 connects to row0,row1 -> chain connects row0,row1,col0,col1.

Stone (1,2): union(row1, col2+offset). Stone (2,1): union(row2, col1+offset). Stone (2,2): union(row2, col2+offset).

All stones connect into one component eventually.

Distinct components containing at least one stone = 1.

`removable = n_stones - n_components = 6 - 1 = 5 ok`

WHY n-components: keep 1 stone per component, remove all others.

DFS version: visited=set(). For each unvisited stone: DFS all stones sharing row or col. visited.add(stone_idx). Each DFS = one component. Removable = total - num_components.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n*alpha(n))	**O(n)** union-find

Follow-up: count distinct connected components — already done (stone_roots).

Follow-up: Number of Connected Components (#323) — same union-find approach.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | DFS | LeetCode #1020 | 20 of 23 | 20/23

Number of Enclaves

QUESTION

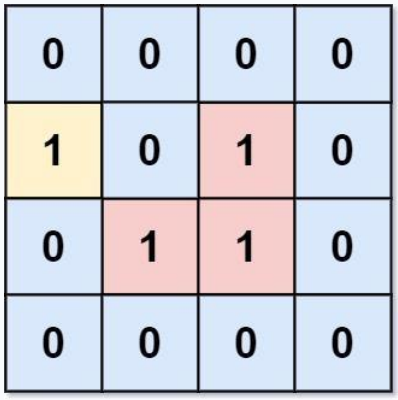
You are given an m x n binary matrix grid, where 0 represents a sea cell and 1 represents a land cell.

A move consists of walking from one land cell to another adjacent (4-directionally) land cell or walking off the boundary of the grid.

Return the number of land cells in grid for which we cannot walk off the boundary of the grid in any number of moves.

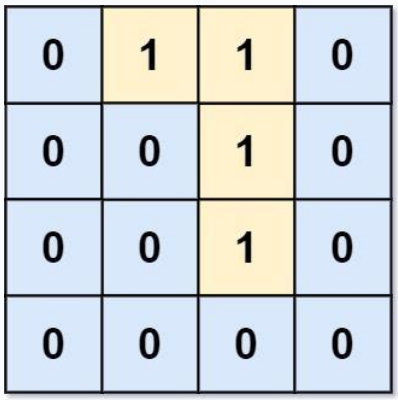
Example 1

Input	`grid = [[0,0,0,0],[1,0,1,0],[0,1,1,0],[0,0,0,0]]`
Output	`3`
Explanation	*There are three 1s that are enclosed by 0s, and one 1 that is not enclosed because its on the boundary.*



Example 2

Input	`grid = [[0,1,1,0],[0,0,1,0],[0,0,1,0],[0,0,0,0]]`
Output	`0`
Explanation	*All 1s are either on the boundary or can reach the boundary.*



CONSTRAINTS

- m == grid.length
- n == grid[i].length
- 1 <= m, n <= 500
- grid[i][j] is either 0 or 1.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Count land cells that cannot reach the boundary by moving 4-directionally through land.

'Enclaves' are land cells completely surrounded by water or blocked from boundary. Right?"

"A few quick questions:

0=water, 1=land? Yes. Boundary = edges of grid? Yes."

"Let me walk: grid=[[0,0,0,0],[1,0,1,0],[0,1,1,0],[0,0,0,0]] -> enclave count = 3 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for cell in cells:
    DFS to see whether its region can reach the
        boundary
    count the cells whose region cannot
```

TIME	SPACE
O((m * n)^2)	O(m * n)

WHY IT IS SLOW

Re-explores the same region from every cell. One DFS or BFS from every boundary land cell marks the reachable land; the unmarked land is the answer: O(m * n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Border-DFS flood then count — already O(mn). Walk-through:"

Step 1: For every '1' on the 4 borders -> DFS to mark all connected '1's as visited.

These are 'escape' land cells — they can reach the boundary.

Step 2: Count all '1' cells NOT marked visited -> these are 'enclaves'.

`dfs(r, c): mark (r,c) visited; recurse into 4 neighbours that are '1' and unvisited.`

HOW the flood marks escape cells: DFS from any border land cell floods its entire

connected component (all land connected to the border), marking every cell.

Cells not flooded after all 4 border DFS runs -> completely surrounded -> enclaves.

PHASE 3 OPTIMIZE

Time: O(mn). Space: O(mn) recursion worst case."

Time: O(log n) — binary search on answer space.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _1020_NumberOfEnclaves:
    def numEnclaves(self, grid):
        m, n = len(grid), len(grid[0])
        def flood(r, c):
            if not (0 <= r < m and 0 <= c < n) or grid[r][
c] != 1:
                return
            grid[r][c] = 0
            for dr, dc in [(0,1),(0,-1),(1,0),(-1,0)]:
                flood(r + dr, c + dc)
        for r in range(m):
            for c in range(n):
                if (r in (0, m-1) or c in (0, n-1)) and gr
id[r][c] == 1:
                    flood(r, c)
        return sum(grid[r][c] for r in range(m) for c in r
ange(n))
```

PHASE 5 TEST & DEBUG

"Let me trace numEnclaves([[0,0,0,0],[1,0,1,0],[0,1,1,0],[0,0,0,0]])."

Step 1: DFS from all border cells, mark reachable land as visited

Border land cells: none on row 0/3 (all 0). Col 0/3: land at (1,0)->DFS marks (1,0). Col 3: none.

Step 2: count remaining unvisited land cells

After border DFS: (1,0) marked. Remaining land: (1,2),(2,1),(2,2) -> count=3 ok

WHY flood fill from borders: land reachable from border = not enclosed. Count the rest.

Edge: no enclosed land -> return 0; island touching border -> all counted as non-enclave.

Same as 'number of islands' but count only interior ones; DFS from border floods reachable land.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(m*n)	**O(m*n)** worst case recursion

Follow-up: Number of Islands (#200) — count connected components of land.

Follow-up: Surrounded Regions (#130) — same boundary-flood idea, replace with 'X'.

MEDIUM | DFS | LeetCode #1236 | 21 of 23 | 21/23

Web Crawler

QUESTION

Given a url startUrl and an interface HtmlParser, implement a web crawler to crawl all links that are under the same hostname as startUrl.

Return all urls obtained by your web crawler in any order.

Your crawler should:

CONSTRAINTS

- Start from the page: startUrl
- Call HtmlParser.getUrls(url) to get all urls from a webpage of given url.
- Do not crawl the same link twice.
- Explore only the links that are under the same hostname as startUrl.



As shown in the example url above, the hostname is example.org. For simplicity sake, you may assume all urls use http protocol without any port specified. For example, the urls http://leetcode.com/problems and http://leetcode.com/contest are under the same hostname, while urls http://example.org/test and http://example.com/abc are not under the same hostname.

The HtmlParser interface is defined as such:

```
interface HtmlParser {
  // Return a list of all urls from a webpage of give
n
url
.
  public List<String> getUrls(String url);
}
```

Below are two examples explaining the functionality of the problem, for custom testing purposes you'll have three variables urls, edges and startUrl. Notice that you will only have access to startUrl in your code, while urls and edges are not directly accessible to you in code.

Note: Consider the same URL with the trailing slash "/" as a different URL. For example, "http://news.yahoo.com", and "http://news.yahoo.com/" are different urls.

QUESTION

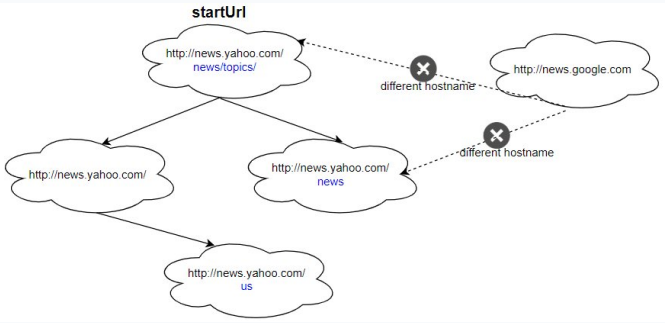
Example 1

Input

```
urls = [
"http://news.yahoo.com",
"http://news.yahoo.com/news",
"http://news.yahoo.com/news/topi
cs/",
"http://news.google.com",
"http://news.yahoo.com/us"
]
edges =
[[2,0],[2,1],[3,2],[3,1],[0,4]]
startUrl = "http://news.yahoo.co
m/news/topics/"
```

Output

```
[
"http://news.yahoo.com",
"http://news.yahoo.com/news",
"http://news.yahoo.com/news/topi
cs/",
"http://news.yahoo.com/us"
]
```



Example 2

Input

```
urls = [
"http://news.yahoo.com",
"http://news.yahoo.com/news",
"http://news.yahoo.com/news/topi
cs/",
"http://news.google.com"
]
edges =
[[0,2],[2,1],[3,2],[3,1],[3,0]]
startUrl =
"http://news.google.com"
```

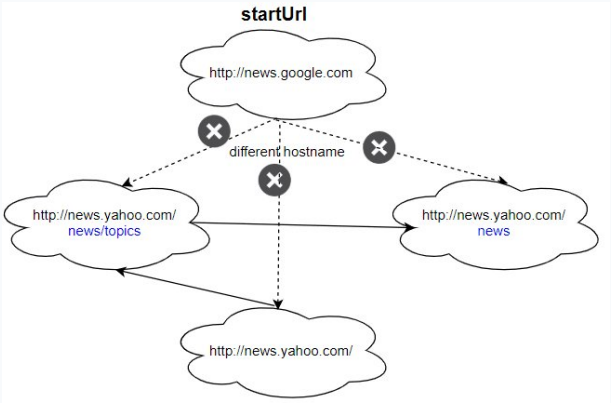
Output

```
["http://news.google.com"]
```

Explanation

The startUrl links to all other pages that do not share the same hostname.

QUESTION



- 1 <= urls.length <= 1000
- 1 <= urls[i].length <= 300
- startUrl is one of the urls.
- Hostname label must be from 1 to 63 characters long, including the dots, may contain only the ASCII letters from 'a' to 'z', digits from '0' to '9' and the hyphen-minus character ('-').
- The hostname may not start or end with the hyphen-minus character ('-').
- See: https://en.wikipedia.org/wiki/Hostname#Restrictions_on_valid_hostnames
- You may assume there're no duplicates in url library.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

BFS from startUrl. Only follow links on the same hostname. Don't revisit. Right?

Hostname = part between '//' and the next '/'."

Is the graph guaranteed to be connected? Can it contain cycles or self-loops?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
visited = a list of urls
crawl from startUrl:
    for link in links:
        # on the same hostname
        if it is not in the list (scan it):
            add it and crawl it
```

TIME	SPACE
O(V^2)	O(V)

WHY IT IS SLOW

Checking the visited list is O(V) per link. A hash set of visited urls makes each check O(1), so the whole crawl is O(V + E) page fetches.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"BFS with hostname filter IS already the right approach — O(V+E)."

Walk-through of how the crawler explores:

Queue starts with startUrl. Pop url -> call htmlParser.getUrls(url) to get all links on that page.

For each link: check hostname matches (same domain only). If unvisited -> add to visited + enqueue.

BFS guarantees we process each page exactly once (visited set).

Hostname extraction: url.split('/')[2] gives the domain.

e.g. 'http://news.yahoo.com/news/1' -> ['http:', '', 'news.yahoo.com', 'news', '1'] -> [2] = 'news.yahoo.com'

Return list(visited) — includes startUrl and all same-domain pages reachable from it."

Time: O(V+E) — visit each vertex and edge exactly once. Optimal DFS/BFS.

PHASE 4 CLEAN CODE

```
class _1236_WebCrawler:
    def crawl(self, startUrl: str, htmlParser: 'HtmlParser
') -> List[str]:
        def hostname(url):
            return url.split('/')[2]   # 'http:' , '' , 'n
ews.yahoo.com', ...

        host    = hostname(startUrl)
        visited = {startUrl}
        queue   = deque([startUrl])

        while queue:
            url = queue.popleft()
            for link in htmlParser.getUrls(url):
                if link not in visited and hostname(link)
== host:
                    visited.add(link)
                    queue.append(link)

        return list(visited)
```

PHASE 5 TEST & DEBUG

"Let me trace web crawler BFS from startUrl='http://news.yahoo.com/news'."

Extract hostname: 'news.yahoo.com'.

BFS from start URL:

queue=['http://news.yahoo.com/news'].
visited={'http://news.yahoo.com/news'}.

Pop URL: htmlParser.getUrls('http://news.yahoo.com/news') -> ['http://news.yahoo.com/news/today','http://m.yahoo.com/'].

Filter same hostname: 'http://m.yahoo.com/' -> hostname 'm.yahoo.com' != 'news.yahoo.com' -> skip.

'http://news.yahoo.com/news/today' -> same hostname -> enqueue.

Continue until no new URLs from same hostname remain.

Result: all crawled URLs with 'news.yahoo.com' hostname ok

WHY BFS: discovers all reachable pages level by level.

PHASE 5 TEST & DEBUG

BFS: visited={'http://news.yahoo.com/news'}. queue=[startUrl]. component: all same-hostname URLs reachable from start = one web component. Pop URL -> getUrls -> filter -> enqueue.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(V+E) where V=pages, E=links	**O(V)**

Standard BFS graph traversal — the only novelty is hostname filtering.

Given more time I'd ask: #1242 Multithreaded? Use a ThreadPoolExecutor,

submit getUrls calls as futures, use a thread-safe visited set.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | DFS | LeetCode #1242 | 22 of 23 | 22/23

Web Crawler Multithreaded

QUESTION

Given a URL startUrl and an interface HtmlParser, implement a Multi-threaded web crawler to crawl all links that are under the same hostname as startUrl.

Return all URLs obtained by your web crawler in any order.

Your crawler should:

CONSTRAINTS

- Start from the page: startUrl
- Call HtmlParser.getUrls(url) to get all URLs from a webpage of a given URL.
- Do not crawl the same link twice.
- Explore only the links that are under the same hostname as startUrl.



As shown in the example URL above, the hostname is example.org. For simplicity's sake, you may assume all URLs use HTTP protocol without any port specified. For example, the URLs http://leetcode.com/problems and http://leetcode.com/contest are under the same hostname, while URLs http://example.org/test and http://example.com/abc are not under the same hostname.

The HtmlParser interface is defined as such:

```
interface HtmlParser {
  // Return a list of all urls from a webpage of give
n
url
.
  // This is a blocking call, that means it will do H
TTP request and return when this request is finished.
  public List<String> getUrls(String url);
}
```

Note that getUrls(String url) simulates performing an HTTP request. You can treat it as a blocking function call that waits for an HTTP request to finish. It is guaranteed that getUrls(String url) will return the URLs within 15ms. Single-threaded solutions will exceed the time limit so, can your multi-threaded web crawler do better?

Below are two examples explaining the functionality of the problem. For custom testing purposes, you'll have three variables urls, edges and startUrl. Notice that you will only have access to startUrl in your code, while urls and edges are not directly accessible to you in code.

QUESTION

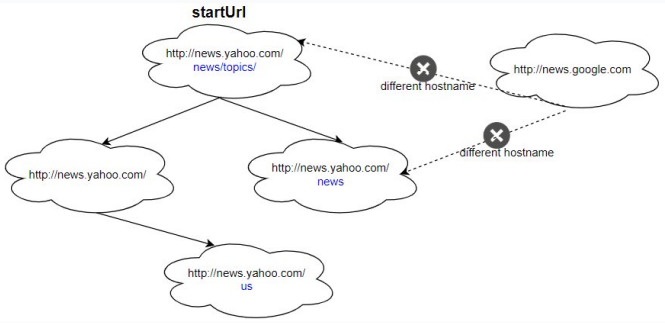
Example 1

Input

```
urls = [
"http://news.yahoo.com",
"http://news.yahoo.com/news",
"http://news.yahoo.com/news/topi
cs/",
"http://news.google.com",
"http://news.yahoo.com/us"
]
edges =
[[2,0],[2,1],[3,2],[3,1],[0,4]]
startUrl = "http://news.yahoo.co
m/news/topics/"
```

Output

```
[
"http://news.yahoo.com",
"http://news.yahoo.com/news",
"http://news.yahoo.com/news/topi
cs/",
"http://news.yahoo.com/us"
]
```



Example 2

Input

```
urls = [
"http://news.yahoo.com",
"http://news.yahoo.com/news",
"http://news.yahoo.com/news/topi
cs/",
"http://news.google.com"
]
edges =
[[0,2],[2,1],[3,2],[3,1],[3,0]]
startUrl =
"http://news.google.com"
```

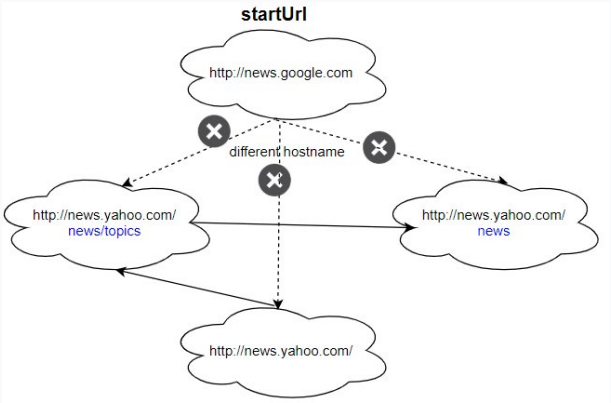
Output

```
["http://news.google.com"]
```

Explanation

The startUrl links to all other pages that do not share the same hostname.

QUESTION



- 1 <= urls.length <= 1000
- 1 <= urls[i].length <= 300
- startUrl is one of the urls.
- Hostname label must be from 1 to 63 characters long, including the dots, may contain only the ASCII letters from 'a' to 'z', digits from '0' to '9' and the hyphen-minus character ('-').
- The hostname may not start or end with the hyphen-minus character ('-').
- See: https://en.wikipedia.org/wiki/Hostname#Restrictions_on_valid_hostnames
- You may assume there're no duplicates in the URL library.
- Follow up:
- Assume we have 10,000 nodes and 1 billion URLs to crawl. We will deploy the same software onto each node. The software can know about all the nodes. We have to minimize communication between machines and make sure each node does equal amount of work. How would your web crawler design change?
- What if one node fails or does not work?
- How do you know when the crawler is done?

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Same as #1236 but with multi-threading because getUrls() is I/O-bound and slow.

Use threads to fetch pages in parallel. Thread-safe visited set required. Right?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
crawl sequentially:
    fetch each page, then its links, one at a
        time, with a visited set
```

TIME	SPACE
O(V + E)	O(V)
fetches in sequence	

WHY IT IS SLOW

PHASE 2 BRUTE FORCE

Waits for each network call before the next. A thread pool fetching pages in parallel with a lock-protected visited set finishes in about (depth * fetch latency) when the web is wide.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Sequential BFS blocks on every I/O call. Use threads so all pending pages"

are fetched in parallel. Key insight: getUrls() is I/O-bound — GIL is released

during the blocking call, so Python threads give real concurrency here.

Use ThreadPoolExecutor to dispatch one thread per URL. Protect visited with a Lock.

PHASE 4 CLEAN CODE (multithreaded BFS)

Now I'll implement with ThreadPoolExecutor for parallel I/O.

```
from concurrent.futures import ThreadPoolExecutor, as_comp
leted
import threading

class _1242_WebCrawlerMultithreaded:
    def crawl(self, startUrl: str, htmlParser) -> List[str
]:
        domain = startUrl.split("/")[2]
        visited = {startUrl}
        lock    = threading.Lock()

        def fetch(url: str) -> List[str]:
            new_urls = []
            for link in htmlParser.getUrls(url):
                if link.split("/")[2] != domain:
                    continue
                with lock:
                    if link not in visited:
                        visited.add(link)
                        new_urls.append(link)
            return new_urls

        frontier = [startUrl]
        with ThreadPoolExecutor(max_workers=16) as pool:
            while frontier:
                futures = {pool.submit(fetch, url): url fo
r url in frontier}
                frontier = []
                for fut in as_completed(futures):
                    frontier.extend(fut.result())
        return list(visited)
```

PHASE 5 TEST & DEBUG

"Let me trace from 'http://news.yahoo.com/news'."

Sequential BFS: queue=[start]. Pop start -> getUrls -> filter domain -> enqueue new.

Multithreaded: same BFS but each frontier batch is fetched in parallel.

WHY lock: visited is shared across threads — race condition without lock.

PHASE 5 TEST & DEBUG

WHY ThreadPoolExecutor: manages thread pool, collects results with Future.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(V + E)	**O(V)**
wall-clock but latency amortized by parallelism	visited + thread pool overhead

Follow-up: distributed crawl — consistent hashing by domain across machines.

IN YOUR APP | Expose | Community

MEDIUM | DFS | LeetCode #1462 | 23 of 23 | 23/23

Course Schedule IV

QUESTION

There are a total of numCourses courses you have to take, labeled from 0 to numCourses - 1. You are given an array prerequisites where prerequisites[i] = [a_i, b_i] indicates that you must take course a_i first if you want to take course b_i.

CONSTRAINTS

- For example, the pair [0, 1] indicates that you have to take course 0 before you can take course 1.

Prerequisites can also be indirect. If course a is a prerequisite of course b, and course b is a prerequisite of course c, then course a is a prerequisite of course c.

You are also given an array queries where queries[j] = [u_j, v_j]. For the jth query, you should answer whether course u_j is a prerequisite of course v_j or not.

Return a boolean array answer, where answer[j] is the answer to the jth query.

Example 1

Input	`numCourses = 2, prerequisites = [[1,0]], queries = [[0,1],[1,0]]`
Output	`[false,true]`
Explanation	*The pair [1, 0] indicates that you have to take course 1 before you can take course 0. Course 0 is not a prerequisite of course 1, but the opposite is true.*



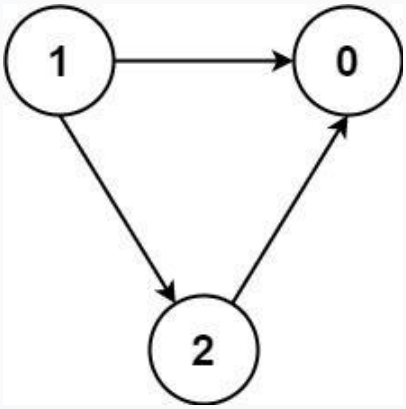
Example 2

Input	`numCourses = 2, prerequisites = [], queries = [[1,0],[0,1]]`
Output	`[false,false]`
Explanation	*There are no prerequisites, and each course is independent.*

Example 3

Input	`numCourses = 3, prerequisites = [[1,2],[1,0],[2,0]], queries = [[1,0],[1,2]]`
Output	`[true,true]`

QUESTION



- 2 <= numCourses <= 100
- 0 <= prerequisites.length <= (numCourses * (numCourses - 1) / 2)
- prerequisites[i].length == 2
- 0 <= a_i, b_i <= numCourses - 1
- a_i != b_i
- All the pairs [a_i, b_i] are unique.
- The prerequisites graph has no cycles.
- 1 <= queries.length <= 10^4
- 0 <= u_i, v_i <= numCourses - 1
- u_i != v_i

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

For each query [ui, vi]: is course ui a prerequisite of vi (directly or indirectly)? Right?"

"A few quick questions:

Transitive prerequisites? Yes. Return bool array for all queries."

"Let me walk: n=2, prerequisites=[[1,0]], queries=[[0,1],[1,0]] -> [False,True] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for b) in b)s:
    run a DFS from a along the prerequisite edges
        and report whether it reaches b
```

TIME	SPACE
O(q * (V + E))	O(V)

WHY IT IS SLOW

A search per query. Precompute the transitive closure once (a DFS from each course, or Floyd-Warshall in O(V^3)) so every query is an O(1) table lookup.

PHASE 2 BRUTE FORCE

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Floyd-Warshall transitive closure — O(V³). Walk-through:"

reachable[i][j] = True if course j is a prerequisite of course i (directly or indirectly).

Build direct edges from prerequisites.

Floyd-Warshall: for k in 0..V-1:

For i,j in all pairs: if reachable[i][k] and reachable[k][j]: reachable[i][j] = True.

For each query (u,v): return reachable[v][u] (is v reachable from u? i.e., v is prereq of u).

Alternative: BFS/DFS from each node -> O(V*(V+E)).

WHY Floyd-Warshall: computes all-pairs reachability in O(V³). Good when many queries.

WHY [v][u] not [u][v]: prereq graph -> edge u->v means u is prereq of v -> reachable[v][u].

Time: O(V³ + Q). Space: O(V²)."

Time: O(n log n) — sorting-based optimal solution.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
from collections import defaultdict, deque
class _1462_CourseScheduleIV:
    def checkIfPrerequisite(self, n, prerequisites, queries):
        reachable = [[False] * n for _ in range(n)]
        indegree = [0] * n
        adj = defaultdict(list)
        for u, v in prerequisites:
            adj[u].append(v)
            indegree[v] += 1
        queue = deque(i for i in range(n) if indegree[i] == 0)
        while queue:
            node = queue.popleft()
            for neighbor in adj[node]:
                reachable[node][neighbor] = True
                for k in range(n):
                    if reachable[k][node]:
                        reachable[k][neighbor] = True
                indegree[neighbor] -= 1
                if indegree[neighbor] == 0:
                    queue.append(neighbor)
        return [reachable[u][v] for u, v in queries]
```

PHASE 5 TEST & DEBUG

"Let me trace this DFS step-by-step, including the call stack and visited marking."

n=2, prerequisites=[[1,0]], queries=[[0,1],[1,0]]:

indegree=[1,0]. queue=[1]. process 1: neighbor=0. reachable[1][0]=True. indegree[0]=0->queue=[0].

process 0: no neighbors. reachable=[...].

queries: reachable[0][1]=False ok. reachable[1][0]=True ok.

PHASE 5 TEST & DEBUG

Graph reachability: dist[u][v]=True if u can reach v. Propagate via topo sort. component: DAG (Directed Acyclic Graph), no strict components. find(u,v)->reachable[u][v]. cost=O(n^2) propagation.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(V² + V*E) O(V) topo sort, O(V²) reachability updates	**O(V²)**

Follow-up: if V is very large, use bitset representation for O(V²/64) space.

Follow-up: All Paths (#797) — return actual paths, not just reachability.